# YupReserved

A hotel reservation system for India

Database Management Systems — project log, report and viva guide

PostgreSQL 16 · 15 tables · 408,201 rows · FastAPI · React

Submitted by: _____ Enrolment no.: _____

Course: _____ Guide: _____

Last updated 29 October 2026, 17:19 · session 3

# Contents

**Part A · Work log** **4**

Session 1 — 29 October 2026: Project foundation: dataset, database, backend, CLI, frontend and this report . . . 4

Session 2 — 29 October 2026: Fix: navigation, Back button, mood cards and 'Try' chips stopped responding . . . 6

Session 3 — 29 October 2026: Photos, HD hero, secure OTP, easier filters, installable mobile app, full-action CLI, deployment . . . 7

**Part B · Project report** **10**

1. Introduction and problem statement . . . 10

2. System architecture and workflow . . . 11

3. Dataset . . . 13

4. Database design . . . 13

5. Keys . . . 17

6. Constraints, triggers, functions and views . . . 20

7. Functional dependencies and normalization . . . 23

8. Relational algebra, calculus and SQL . . . 27

9. Retrieval: what the website asks the database . . . 40

10. Query processing and optimization . . . 44

11. Transactions, concurrency control and recovery . . . 50

12. Distributed, object-oriented, multimedia databases, data mining and digital libraries . . . 52

13. Interfaces: API, CLI and website . . . 56

14. How to run . . . 56

**Part C · Viva questions and answers** **57**

Design and ER model . . . 57

Keys . . . 57

Constraints and triggers . . . 58

Normalization . . . 59

Queries, algebra and calculus . . . 59

Query processing . . . 60

Transactions, concurrency and recovery . . . 60

Dataset, system and Unit 5 . . . 61

**Part D · Syllabus coverage** **63**

Unit 1 · Introduction . . . 63

Unit 2 · Relational model . . . 63

Unit 3 · Relational database design . . . 63

Unit 4 · Query processing, optimization and tuning . . . 63

Unit 5 · Transactions, concurrency, recovery and new applications . . . . . 64

# Part A · Work log

This part is updated after every working session, so the whole history of the project can be traced: what was built, which files were added, which tables and concepts were involved, and why each decision was taken.

## Session 1 — 29 October 2026: Project foundation: dataset, database, backend, CLI, frontend and this report

Built the complete YupReserved system from an empty folder. A real public dataset of 4,000 Indian hotel listings was downloaded, cleaned and normalised into a 15-table PostgreSQL database with 408,192 rows; a FastAPI backend, a terminal CLI and a React frontend were built on top of it, and every syllabus unit was mapped to a working, demonstrable feature.

### Decisions and why

**PostgreSQL 16 instead of the installed MySQL 5.7.** MySQL 5.7 parses CHECK constraints but does not enforce them, and has no FULL OUTER JOIN, INTERSECT/EXCEPT, CTEs or window functions — all needed for the syllabus. PostgreSQL also offers EXCLUDE constraints, deferred constraint triggers, table inheritance (OODB), partitioning (distributed fragments), full-text search (digital libraries) and readable EXPLAIN ANALYZE plans.

**Goibibo hotel listings (Kaggle, PromptCloud) as the base dataset.** 4,000 real Indian properties with city, state, star rating, coordinates, facilities, nearby points of interest and room counts. It contains genuine 1NF violations (pipe-separated lists) and inconsistent state spellings, which make normalisation demonstrable on real data.

**Prices generated by a documented pricing model.** No public dataset of Indian hotels includes nightly prices (the Cities42 price file is not publicly downloadable). Base prices are computed from star rating, property type and city, and every booking is priced by the same SQL function the website uses. This is stated openly in the dataset README.

**Exactly 15 core tables, teaching objects kept in separate schemas.** The 15 business tables live in schema public. Raw data (schema staging) and demonstrations (schema lab: normal-form walk-through, OODB, fragments, multimedia, transaction demo table) never mix with business data.

**Surrogate room_id, and rooms left in 3NF (not BCNF) on purpose.** With room_type_id → hotel_id, BCNF decomposition would lose the dependency (hotel_id, room_number) → room_id. 3NF keeps every dependency enforceable; a composite foreign key keeps room and room type in the same hotel.

**Raw SQL, no ORM; every API response carries its SQL.** So that every screen can be explained in the viva: the frontend's 'Under the hood' drawer shows the exact SQL, rows and timing behind each page.

**Server and Python/Node tooling stored outside the project.** PostgreSQL binaries, data directory and the Python virtual environment live in ~/.yupreserved so the project folder stays clean and presentable in the file explorer.

### Files added or changed

| file / folder | what it is |
|---|---|
| 01-dataset/raw/goibibo_com-travel_sample.csv | the untouched source dataset (4,000 listings, 36 columns) |

| file / folder | what it is |
|---|---|
| 01-dataset/reference/*.csv, signature_stays.json | states (ISO-style codes), festival seasons, 95 curated landmarks with coordinates, 18 original YupReserved stays |
| 01-dataset/pipeline/vocab.py | cleaning vocabularies: state aliases, city fixes, chain-brand filter, 356 → 52 amenity mapping |
| 01-dataset/pipeline/build_dataset.py | clean → enrich → simulate pipeline producing one CSV per table |
| 01-dataset/processed/*.csv | 14 clean CSV files, 329,508 rows (+ manifest and amenity mapping) |
| 02-database/build.sql | rebuilds the whole database in one command |
| 02-database/schema/01…07_*.sql | domains & enums, 15 tables, functions/procedures, 8 triggers, indexes, views, roles |
| 02-database/seed/load_data.sql | bulk load with \copy; prices and payments derived in SQL |
| 02-database/lab/lab_objects.sql | raw staging table, UNF→3NF tables, 4NF/5NF demos, OODB, fragments, media, txn demo |
| 02-database/queries/01…07_*.sql | 66 named, annotated queries (relational algebra + explanation) |
| 02-database/demos/why_not.sql, query_processing.sql | 16 constraint scenarios and 15 query-plan demonstrations |
| 03-backend/app/* | FastAPI app: db layer, catalog parser, error explainer, lab engine, concurrency demos, 3 routers |
| 04-cli/yup.py and ./yup | terminal tool: tables, show, describe, keys, q, explain, why, txn, quote, find, sql, recovery |
| 05-frontend/src/* | React + TypeScript site: poster-art engine, 8 pages, design system, EN/□□□, light/dark |
| 06-docs/diagrams/* | ER (Chen), relational schema, architecture and workflow diagrams (Graphviz) |
| 06-docs/report/* | this report: work log + builder |
| scripts/db.sh | start / stop / build / psql helper for the local PostgreSQL server |

## Tables involved

states, cities, landmarks, hotels, hotel_landmarks, amenities, hotel_amenities, room_types, rooms, seasons, guests, bookings, booking_rooms, payments, reviews

## DBMS concepts used

- ER model: strong & weak entities, partial key, 1:1 / 1:N / M:N, recursive relationship, multivalued & derived attributes
- Keys: super, candidate, primary, alternate, composite, surrogate, natural, foreign (incl. self-referencing and composite)
- Integrity: domains, NOT NULL, CHECK, UNIQUE, EXCLUDE, referential actions (RESTRICT / CASCADE / SET NULL)
- Triggers (BEFORE / AFTER / deferred constraint trigger), stored functions and a stored procedure
- Views, updatable view WITH CHECK OPTION, security view, materialized view
- Relational algebra (σ π ρ ⋈ ∪ ∩ − ÷ γ), tuple and domain relational calculus
- All joins: inner, natural, theta, left/right/full outer, self, cross, semi, anti, lateral, multi-way
- Functional dependencies, 1NF → 2NF → 3NF → BCNF, MVD / 4NF, join dependency / 5NF, lossless join
- Query processing: seq/index/bitmap scans, hash/merge/nested-loop joins, external sort, pipelining, heuristics

- Transactions: ACID, 2PL row locks, deadlock detection, isolation levels, MVCC, WAL crash recovery
- Distributed (fragmentation), object-relational (types, inheritance), multimedia (bytea), data mining, full-text search
- DCL: roles, GRANT / REVOKE, least privilege, SECURITY DEFINER

### Verified

- Database rebuilt from scratch with every trigger and constraint active: all 408,192 rows accepted.
- All 66 named queries run without error (all under 100 ms).
- All 16 'why not' scenarios are refused by the intended rule.
- All 15 query-plan demonstrations show the intended operator (e.g. external merge sort, merge join).
- All 5 concurrency demonstrations behave as expected (lock wait, deadlock victim, non-repeatable read, MVCC, atomicity).
- End-to-end booking, double-cancel refusal and capacity rejection tested through the website's API.
- Crash recovery demonstrated (./yup recovery): committed row survived, uncommitted row gone; API reconnects automatically.
- Lossless-join check on the cleaned data: 57,088 rows in, 57,088 out, 0 spurious (raw data was lossy: 357 rows).
- Bugs found and fixed during testing: double cancellation was allowed (now refused by sp_cancel_booking); window.scrollTo returning a Promise broke React effects; lab scenarios hit privileges instead of constraints (yup_lab role added).
- Frontend type-checks and builds for production (~140 kB gzipped JavaScript).

## Session 2 — 29 October 2026: Fix: navigation, Back button, mood cards and 'Try' chips stopped responding

The user reported that the Back button, the header links (Stays, My trips, Under the hood), the mood cards and the 'Try' chips did nothing. Reproduced in a real Chrome browser with an automated test, found the single root cause, fixed it in three layers, and verified every link and a full booking end to end.

### Decisions and why

**Root cause: one page error crashed the whole app.** Opening /stays without dates (every one of those links) passed 'undefined' dates into the search bar, which replaced the default dates; date arithmetic then threw 'Invalid time value'. With no error boundary, React unmounted the entire app, so no link or the Back button could respond any more.

**Fix 1 — the cause.** The search bar now ignores unset initial values, and the date helpers (fromISO/addDays) fall back to the platform's 'today' instead of producing an Invalid Date.

**Fix 2 — a safety net.** An ErrorBoundary around the pages, keyed by the URL path: if a page ever fails, a friendly message appears, while the header, Back and every other page keep working.

**Fix 3 — feedback that survives a re-render.** After cancelling, a trip moves from 'Coming up' to 'Past & cancelled' and its card is re-created, which erased its success message. The message is now a page-level banner.

### Files added or changed

| file / folder | what it is |
|---|---|
| 05-frontend/src/components/search.tsx | ignore undefined initial values |
| 05-frontend/src/lib/format.ts | safe date parsing |
| 05-frontend/src/components/ErrorBoundary.tsx | new — page-level error boundary |
| 05-frontend/src/App.tsx | routes wrapped in the error boundary |
| 05-frontend/src/pages/Trips.tsx | page-level notice for cancellations and reviews |
| 05-frontend/src/components/feedback.tsx | empty state can be the page's main heading (h1) |
| 05-frontend/src/art/Poster.tsx | missing React list key |

### Tables involved

bookings, payments (two test bookings created and cancelled through the website)

### DBMS concepts used

- Frontend reliability: error boundaries, defensive parsing of URL parameters
- The cancellation path exercised the stored procedure sp_cancel_booking and the refund trigger

### Verified

- Automated browser test: all 15 entry points (3 header links, 8 mood cards, 4 'Try' chips) open their page and Back returns home — 15/15.
- Stays → hotel → Back → Back works.
- Full booking in the browser: hotel → Reserve → OTP sign-in → UPI → confirmation → My trips → cancel with refund banner.
- Type-check and production build pass.

## Session 3 — 29 October 2026: Photos, HD hero, secure OTP, easier filters, installable mobile app, full-action CLI, deployment

Added real photos for 97% of hotels (Wikimedia Commons, free licences, credited), a rotating HD hero of 12 Indian landmarks, a genuine OTP sign-in with hashed codes, expiry, attempt and rate limits and signed session tokens, redesigned filters, a Progressive Web App with a phone tab bar, a CLI that can perform every action of the website, and one-container deployment files.

### Decisions and why

**Photos of the place, not of the hotel.** The dataset has no hotel photos, and other sites' photos are copyrighted. Each hotel shows its nearest photographed landmark within 4 km, otherwise one of its city's photos (chosen by hotel_id for variety), always captioned and credited. Photos are hot-linked from Wikimedia, so nothing heavy is stored.

**Photo columns, not a photo table.** photo_url, photo_url_hd and photo_credit are attributes of cities and landmarks (each FD city_id → photo holds), so the core stays at exactly 15 tables. The view v_hotel_photo derives a hotel's photo.

**A NULL-logic bug caught by the database.** The first CHECK (photo_url ~ '^https://' AND photo_credit IS NOT NULL) rejected cities WITHOUT a photo: NULL AND FALSE is FALSE. Fixed to (photo_url IS NULL OR (…)). A real example of SQL three-valued logic.

**OTP done properly.** Codes come from a CSPRNG, only an HMAC-SHA256 is stored (schema auth, outside the 15 tables), 5-minute expiry, 5 attempts, 30 s resend cooldown, 3 codes per 10 minutes, SELECT … FOR UPDATE on the challenge. Sign-in returns an HMAC-signed token; bookings, trips, cancellations and reviews now check the token's guest, closing a hole where any guest_id could be sent. SMS providers (Fast2SMS, Twilio) plug in through environment variables; without one, demo mode shows the code on screen.

**Progressive Web App instead of a separate native app.** One codebase installs on Android and iPhone home screens (manifest, icons, service worker with offline cache), with a native-style bottom tab bar and safe-area support.

**One container for deployment.** FastAPI serves the built React site from the same origin (no CORS), packaged in a Dockerfile with a Render blueprint; deploy-db.sh copies the database to any hosted PostgreSQL with fresh random role passwords.

## Files added or changed

| file / folder | what it is |
|---|---|
| 01-dataset/pipeline/fetch_photos.py, reference/photos.csv | Wikipedia/Commons API fetcher (batched, polite) and its cached result |
| 02-database/schema/02_tables.sql, 06_views.sql | photo columns + CHECKs, view v_hotel_photo |
| 02-database/schema/08_auth.sql | schema auth: otp_challenges |
| 03-backend/app/security.py | OTP generation/hashing, SMS providers, signed session tokens |
| 03-backend/app/routers/booking_routes.py | real OTP endpoints, ownership checks on every guest action |
| 03-backend/app/main.py | serves the built website in production |
| 04-cli/yup.py, 04-cli/README.md | search, hotel, avail, signup, book, booking, cancel, review, guest, trips, stats, menu, install, sql --write |
| 05-frontend/src/components/Photo.tsx, AuthDialog.tsx | photo with poster fallback, HD hero slideshow; new sign-in dialog |
| 05-frontend/src/pages/Search.tsx | redesigned filters |
| 05-frontend/public/manifest.webmanifest, sw.js, icons/ | installable app |
| Dockerfile, .dockerignore, render.yaml, scripts/deploy-db.sh | deployment |

## Tables involved

cities (+photo columns), landmarks (+photo columns), guests, bookings, payments, reviews, auth.otp_challenges

## DBMS concepts used

- Three-valued logic in CHECK constraints (NULL AND FALSE = FALSE)
- Derived data through a view with LATERAL joins (v_hotel_photo)
- Security: hashing secrets, constant-time comparison, rate limiting, authorization by ownership
- Row locking (FOR UPDATE) on the OTP challenge so two verifications cannot race

## Verified

- Database rebuilt from scratch with the new columns and schema: all rows accepted.

- OTP: wrong code counts attempts, codes cannot be reused, resend cooldown enforced, tampered token rejected, a guest cannot read or cancel another guest's bookings.
- Browser test: full booking with the new OTP (sign in → UPI → confirmation → cancel) and 15/15 navigation checks.
- CLI: search, hotel, avail, signup (and duplicate refused), book, booking, review (refused before stay), cancel, guest, menu.
- Production mode: FastAPI serves the built site, manifest, service worker and API from one origin.

# Part B · Project report

## 1. Introduction and problem statement

Travellers in India book everything from a sea-facing heritage hotel in Mumbai to a ₹1,500 guest house near the ghats of Varanasi. Behind every booking a platform must answer hard questions instantly and correctly: Which rooms are free for these dates? What does each night cost during Diwali? Can two people book the last room at the same moment? **YupReserved** is a hotel reservation platform whose database answers these questions with relational guarantees instead of application code.

| problem | how YupReserved solves it |
|---|---|
| Listing data is messy: lists in one cell, inconsistent state names | A cleaning pipeline normalises the real dataset into 15 tables (up to BCNF / 4NF, rooms deliberately 3NF) |
| A room must never be sold twice for overlapping nights | Trigger trg_booking_rooms_guard + row locks (SELECT ... FOR UPDATE) in fn_book_stay |
| Prices change with festivals, weekends and GST slabs | seasons table (EXCLUDE: no overlaps) + fn_quote_stay computes every night |
| Invalid data (check-out before check-in, bad phone numbers, under-age guests) | Domains, CHECK, UNIQUE and foreign keys enforced by the DBMS itself |
| Users need to know why an action was refused | The API translates the violated constraint into a plain-language rule |
| Reports need joins across many tables | Views (v_hotel_cards, v_booking_summary) and a materialized view |

### 1.1 Technologies

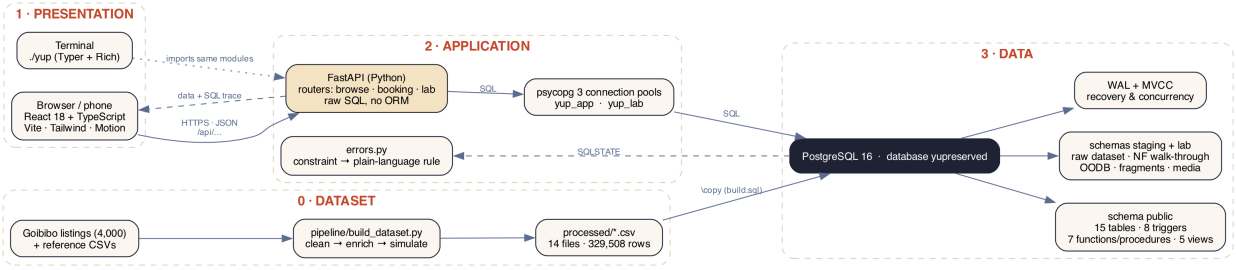
| layer | technology | why |
|---|---|---|
| Database | PostgreSQL 16 | full SQL standard support, enforced CHECK, EXCLUDE, deferred constraints, MVCC, WAL, EXPLAIN ANALYZE |
| Dataset pipeline | Python 3.13, pandas | repeatable, seeded cleaning and simulation |
| Backend | FastAPI, psycopg 3, connection pool | small, fast, typed; raw SQL so every query is visible |
| CLI | Typer + Rich | retrieve tables and run every demo from the terminal |
| Frontend | React 18, TypeScript, Vite, Tailwind CSS 4, Motion, TanStack Query | fast to build, small static bundle, deploys to any static host |
| Diagrams / report | Graphviz, ReportLab | diagrams and this report are generated from the live catalog |

### 1.2 Basic concepts used

| term | meaning | where in YupReserved |
|---|---|---|
| Relation (table) | a set of tuples with the same attributes | 15 tables in schema public |
| Primary key | minimal attribute set that identifies each tuple; never NULL | every table, e.g. bookings(booking_id) |
| Candidate / alternate key | another minimal unique identifier not chosen as PK | guests.phone, guests.email, bookings.booking_ref |
| Foreign key | attribute(s) that must match a key of another (or the same) table | 17 FKs, e.g. bookings.hotel_id → hotels |
| Domain | named type with a rule | indian_mobile, email_address, rupees |
| Trigger | event → condition → action rule inside the DBMS | 8 triggers (04_triggers.sql) |
| View | stored query, virtual table | v_hotel_cards, v_booking_summary, v_guest_directory |
| Transaction (ACID) | all-or-nothing unit of work | fn_book_stay, sp_cancel_booking |

# 2. System architecture and workflow

YupReserved is a **three-tier** system with a fourth, offline data tier that prepares the dataset. The browser never talks to the database: it calls the FastAPI application over HTTP/JSON, and only the application holds database credentials. The CLI reuses the backend's own modules, so the CLI, the API and the website run the same SQL.

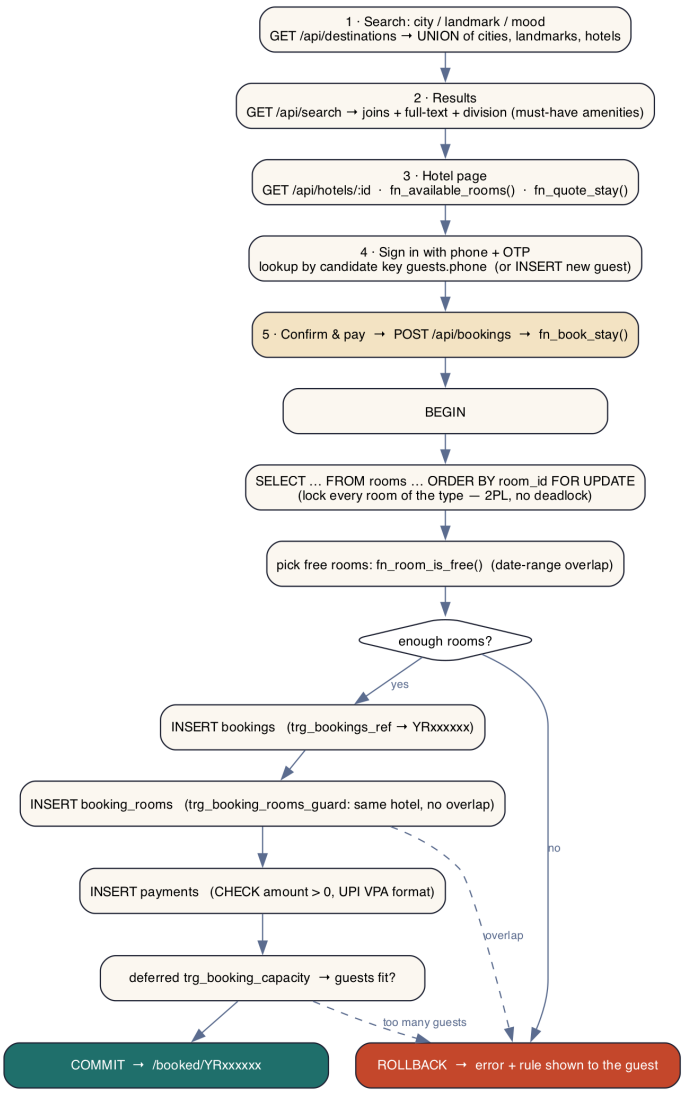



## Why this architecture

- **Separation of concerns** — presentation, business logic and data can be changed independently (data independence at the application level).
- **Security** — the browser has no SQL access; the API connects as yup_app, a least-privilege role that cannot DELETE bookings or touch rooms directly.
- **Rules live in the database** — constraints and triggers protect the data whichever program writes to it (website, CLI or psql).
- **Transparency** — every API response carries the SQL it ran, which the website shows in its 'Under the hood' drawer.
- **Laptop-friendly** — PostgreSQL runs with 128 MB shared buffers and 30 connections; the frontend is static files.

## 2.1 Booking workflow

The workflow below follows one guest from search to a committed booking. Steps 5–9 run inside a single database transaction in the function fn_book_stay.


1 · Search: city / landmark / mood
GET /api/destinations → UNION of cities, landmarks, hotels
2 · Results
GET /api/search → joins + full-text + division (must-have amenities)
3 · Hotel page
GET /api/hotels/:id · fn_available_rooms() · fn_quote_stay()
4 · Sign in with phone + OTP
lookup by candidate key guests.phone (or INSERT new guest)
5 · Confirm & pay → POST /api/bookings → fn_book_stay()
BEGIN
SELECT … FROM rooms … ORDER BY room_id FOR UPDATE
(lock every room of the type — 2PL, no deadlock)
pick free rooms: fn_room_is_free() (date-range overlap)
enough rooms?
yes
no
INSERT bookings (trg_bookings_ref → YRxxxxxx)
INSERT booking_rooms (trg_booking_rooms_guard: same hotel, no overlap)
overlap
INSERT payments (CHECK amount > 0, UPI VPA format)
deferred trg_booking_capacity → guests fit?
too many guests
COMMIT → /booked/YRxxxxxx
ROLLBACK → error + rule shown to the guest

# 3. Dataset

**Source:** 'Indian Hotels on Goibibo' (PromptCloud, published on Kaggle; mirrored on Hugging Face as jason1966/PromptCloudHQ_hotels-on-goibibo): 4,000 listings, 36 columns, crawled in 2016. It is kept untouched in 01-dataset/raw/ and also loaded as-is into staging.goibibo_raw for the normalisation walk-through.

| cleaning rule | rows |
|---|---|
| C0 raw rows | 4000 |
| C1 duplicate (name, city) removed | 1 |
| C2 coordinates outside India removed | 3 |
| C3 chain-brand listings removed | 431 |
| C4 unknown state / property type removed | 0 |
| C5 clean listings kept | 3565 |

Other cleaning: state names normalised to 2-letter codes (e.g. 'ANDHRA PRADESH', 'Andhra Pradesh' → AP; 'Orissa' → OD); 2016-era filings corrected (Hyderabad → Telangana, Leh → Ladakh, Chandigarh → its own UT); 'Cochin' / 'Ernakulam' → Kochi; 356 raw facility labels mapped to 52 canonical amenities (processed/_amenity_mapping.csv); check-in/out times parsed from the policy text; HTML entities decoded.

**Enrichment and simulation.** The listings give one room type and a room count per hotel, but no prices, guests or bookings. The pipeline (seed 2026, so it is reproducible) adds room-type tiers, physical rooms, a festival calendar, 26,000 guests, 72,000 bookings and 19,041 reviews. Base prices follow a documented model (star rating × property type × city factor × review quality, with noise); nightly prices and all payments are then computed inside PostgreSQL. Resulting base-price quartiles: ₹1,699 / ₹2,699 / ₹5,299. Eighteen 'Signature' hotels are original YupReserved properties written for the platform.

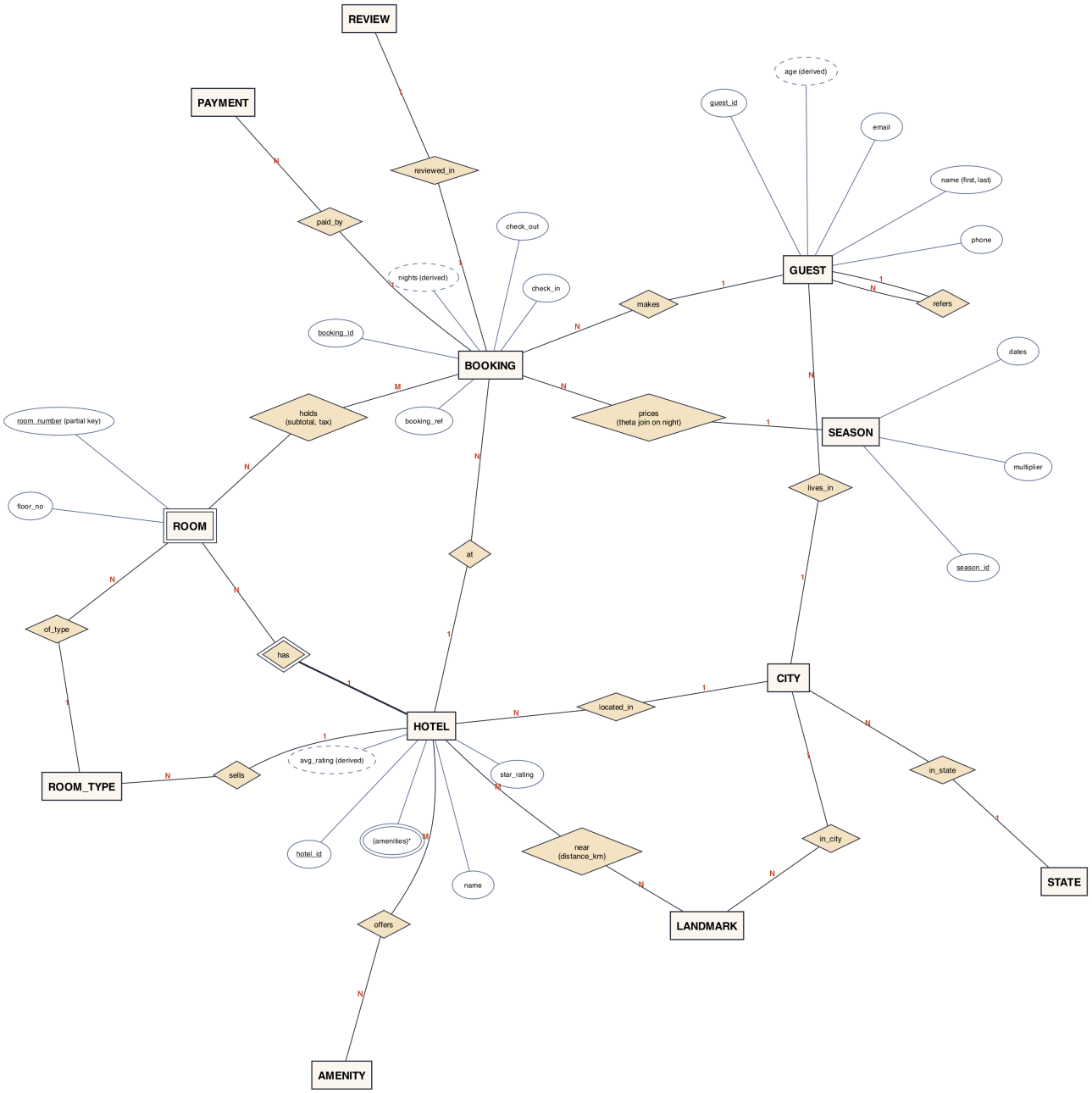
## 3.1 Rows per table

| table_name | rows |
|---|---|
| seasons | 16 |
| states | 36 |
| amenities | 52 |
| landmarks | 2056 |
| cities | 543 |
| hotels | 3583 |
| room_types | 8626 |
| hotel_landmarks | 17556 |
| reviews | 19041 |
| guests | 26001 |
| hotel_amenities | 38107 |
| rooms | 58511 |
| payments | 78688 |
| bookings | 72002 |
| booking_rooms | 83383 |

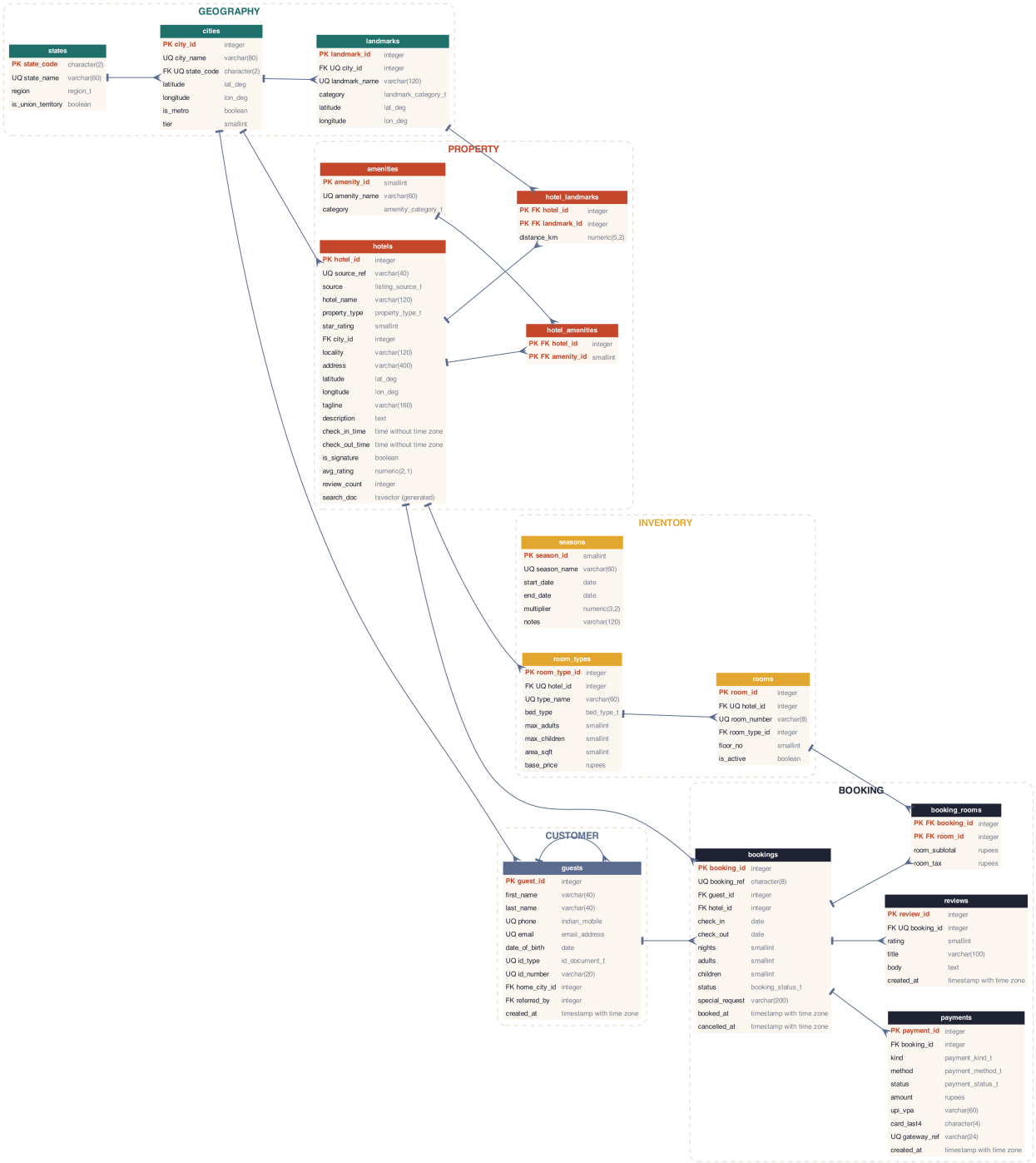
# 4. Database design

## 4.1 Conceptual model — ER diagram (Chen notation)

Rectangles are entities, diamonds relationships, ellipses attributes. ROOM is a **weak entity** (double box) identified through HOTEL by the partial key room_number; 'has' is its identifying relationship. {amenities} is a multivalued attribute (double ellipse), dashed ellipses are derived attributes, and 'refers' is a recursive relationship on GUEST.



## 4.2 Logical model — relational schema (generated from the live catalog)



## 4.3 ER → relational mapping

| ER construct | mapped to | example |
|---|---|---|
| Strong entity | own table, own PK | HOTEL → hotels(hotel_id) |
| Weak entity + identifying relationship | table with owner's key + partial key | ROOM → rooms: candidate key (hotel_id, room_number); surrogate room_id as PK |
| 1:N relationship | FK on the N side | located_in → hotels.city_id |
| M:N relationship | separate table, composite PK of both keys | offers → hotel_amenities(hotel_id, amenity_id) |
| M:N with attributes | attributes go into the relationship table | near(distance_km) → hotel_landmarks.distance_km; holds(subtotal, tax) → booking_rooms |
| 1:1 relationship | FK + UNIQUE | reviewed_in → reviews.booking_id UNIQUE |
| Recursive relationship | self-referencing FK | refers → guests.referred_by → guests.guest_id |
| Multivalued attribute | separate table (1NF) | {amenities} → hotel_amenities |
| Composite attribute | one column per component | name → first_name, last_name |
| Derived attribute | computed, or stored and maintained | nights GENERATED; age in a view; avg_rating kept by trigger |

## 4.4 The 15 tables

| # | table | stores | primary key | references |
|---|---|---|---|---|
| 1 | states | Indian states and union territories (natural key) | state_code | — |
| 2 | cities | cities, each in one state | city_id | states |
| 3 | landmarks | places guests search by | landmark_id | cities |
| 4 | hotels | one row per property | hotel_id | cities |
| 5 | hotel_landmarks | which landmarks are near which hotel, with distance | hotel_id, landmark_id | hotels, landmarks |
| 6 | amenities | controlled amenity vocabulary | amenity_id | — |
| 7 | hotel_amenities | which hotel offers which amenity | hotel_id, amenity_id | amenities, hotels |
| 8 | room_types | what a hotel sells, with list price | room_type_id | hotels |
| 9 | rooms | physical rooms (weak entity of hotel) | room_id | room_types |
| 10 | seasons | festival / holiday pricing calendar | season_id | — |
| 11 | guests | customers | guest_id | cities, guests |
| 12 | bookings | reservation header: guest, hotel, dates, status | booking_id | guests, hotels |
| 13 | booking_rooms | rooms held by a booking, with agreed price | booking_id, room_id | bookings, rooms |
| 14 | payments | charges and refunds | payment_id | bookings |
| 15 | reviews | one review per completed stay | review_id | bookings |

# 5. Keys

**Super key**: any attribute set that uniquely identifies a tuple (e.g. {booking_id, guest_id}). **Candidate key**: a minimal super key. **Primary key**: the candidate key chosen to identify rows; the others become **alternate keys** enforced with UNIQUE. A **surrogate** key is a system-generated number with no business meaning; a **natural** key comes from the real world.

## Primary keys of all 15 tables [catalog.primary_keys]

RA / RC: — (catalog)

11 single-column surrogate/natural keys and 4 composite keys (on the M:N relationship tables).

```
SELECT con.conrelid::regclass::text AS table_name, con.conname AS constraint_name,
       string_agg(a.attname, ', ' ORDER BY k.ord) AS key_columns,
       CASE WHEN count(*) > 1 THEN 'composite' ELSE 'single' END AS kind
FROM pg_constraint con
JOIN pg_namespace n ON n.oid = con.connamespace
CROSS JOIN LATERAL unnest(con.conkey) WITH ORDINALITY AS k(attnum, ord)
JOIN pg_attribute a ON a.attrelid = con.conrelid AND a.attnum = k.attnum
WHERE con.contype = 'p' AND n.nspname = 'public'
GROUP BY con.conrelid, con.conname
ORDER BY kind DESC, table_name
```

| table_name | constraint_name | key_columns | kind |
|---|---|---|---|
| amenities | pk_amenities | amenity_id | single |
| bookings | pk_bookings | booking_id | single |
| cities | pk_cities | city_id | single |
| guests | pk_guests | guest_id | single |
| hotels | pk_hotels | hotel_id | single |
| landmarks | pk_landmarks | landmark_id | single |
| payments | pk_payments | payment_id | single |
| reviews | pk_reviews | review_id | single |
| room_types | pk_room_types | room_type_id | single |
| rooms | pk_rooms | room_id | single |
| seasons | pk_seasons | season_id | single |
| states | pk_states | state_code | single |
| booking_rooms | pk_booking_rooms | booking_id, room_id | composite |
| hotel_amenities | pk_hotel_amenities | hotel_id, amenity_id | composite |
| hotel_landmarks | pk_hotel_landmarks | hotel_id, landmark_id | composite |

| table | why this primary key |
|---|---|
| states (state_code) | natural key: 2-letter codes are short, stable and universally used (MH, RJ), so no surrogate is needed |
| cities, landmarks, hotels, room_types, guests, bookings, payments, reviews (…_id) | surrogate integer identity: natural candidates (names, phone, email) can change or be long; a small integer makes joins and FKs fast and … |
| rooms (room_id) | surrogate PK; the natural candidate key (hotel_id, room_number) is kept as UNIQUE. A single-column key keeps booking_rooms in 2NF |
| amenities, seasons (smallint) | small lookup tables: 2-byte keys are enough |
| hotel_amenities, hotel_landmarks, booking_rooms (composite) | M:N relationship tables: the pair of foreign keys IS the identity; a pair can appear only once |

## Alternate (candidate) keys — every UNIQUE constraint [catalog.candidate_keys]

RA / RC: — (catalog)

A candidate key that was not chosen as the primary key is an alternate key, enforced with UNIQUE.

```
SELECT con.conrelid::regclass AS table_name, con.conname AS constraint_name,
```

```
       pg_get_constraintdef(con.oid) AS definition
FROM pg_constraint con JOIN pg_namespace n ON n.oid = con.connamespace
WHERE con.contype = 'u' AND n.nspname = 'public'
ORDER BY 1, 2
```

| table_name | constraint_name | definition |
|---|---|---|
| states | uq_states_name | UNIQUE (state_name) |
| cities | uq_cities_name_state | UNIQUE (city_name, state_code) |
| landmarks | uq_landmarks_city_name | UNIQUE (city_id, landmark_name) |
| hotels | uq_hotels_source_ref | UNIQUE (source_ref) |
| amenities | uq_amenities_name | UNIQUE (amenity_name) |
| room_types | uq_room_types_hotel_name | UNIQUE (hotel_id, type_name) |
| room_types | uq_room_types_id_hotel | UNIQUE (room_type_id, hotel_id) |
| rooms | uq_rooms_hotel_number | UNIQUE (hotel_id, room_number) |
| seasons | uq_seasons_name | UNIQUE (season_name) |
| guests | uq_guests_email | UNIQUE (email) |
| guests | uq_guests_id_document | UNIQUE (id_type, id_number) |
| guests | uq_guests_phone | UNIQUE (phone) |
| bookings | uq_bookings_ref | UNIQUE (booking_ref) |
| payments | uq_payments_gateway_ref | UNIQUE (gateway_ref) |
| reviews | uq_reviews_booking | UNIQUE (booking_id) |

## Foreign keys — child → parent, with ON DELETE rule [catalog.foreign_keys]

RA / RC: — (catalog)

Every arrow of the ER diagram is one row here. ON DELETE says what happens to children when a parent row is deleted.

```
SELECT con.conrelid::regclass AS child_table,
       pg_get_constraintdef(con.oid) AS definition,
       con.conname AS constraint_name,
       CASE con.confdeltype WHEN 'a' THEN 'NO ACTION' WHEN 'r' THEN 'RESTRICT'
            WHEN 'c' THEN 'CASCADE' WHEN 'n' THEN 'SET NULL' WHEN 'd' THEN 'SET DEFAULT' END AS on_delete
FROM pg_constraint con
JOIN pg_namespace n ON n.oid = con.connamespace
WHERE con.contype = 'f' AND n.nspname = 'public'
ORDER BY 1, 3
```

| child_table | definition | constraint_name | on_delete |
|---|---|---|---|
| cities | FOREIGN KEY (state_code) REFERENCES states(state_code) ON UPDATE CASCADE ON DELETE RESTRICT | fk_cities_states | RESTRICT |
| landmarks | FOREIGN KEY (city_id) REFERENCES cities(city_id) ON DELETE CASCADE | fk_landmarks_cities | CASCADE |
| hotels | FOREIGN KEY (city_id) REFERENCES cities(city_id) ON DELETE RESTRICT | fk_hotels_cities | RESTRICT |
| hotel_landmarks | FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id) ON DELETE CASCADE | fk_hotel_landmarks_hotels | CASCADE |
| hotel_landmarks | FOREIGN KEY (landmark_id) REFERENCES landmarks(landmark_id) ON DELETE CASCADE | fk_hotel_landmarks_landmarks | CASCADE |
| hotel_amenities | FOREIGN KEY (amenity_id) REFERENCES amenities(amenity_id) ON DELETE RESTRICT | fk_hotel_amenities_amenities | RESTRICT |
| hotel_amenities | FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id) ON DELETE CASCADE | fk_hotel_amenities_hotels | CASCADE |
| room_types | FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id) ON DELETE CASCADE | fk_room_types_hotels | CASCADE |
| rooms | FOREIGN KEY (room_type_id, hotel_id) REFERENCES room_types(room_type_id, hotel_id) ON DELETE CASCADE | fk_rooms_room_types | CASCADE |

| child_table | definition | constraint_name | on_delete |
|---|---|---|---|
| guests | FOREIGN KEY (home_city_id) REFERENCES cities(city_id) ON DELETE SET NULL | fk_guests_cities | SET NULL |
| guests | FOREIGN KEY (referred_by) REFERENCES guests(guest_id) ON DELETE SET NULL | fk_guests_referrer | SET NULL |
| bookings | FOREIGN KEY (guest_id) REFERENCES guests(guest_id) ON DELETE RESTRICT | fk_bookings_guests | RESTRICT |
| bookings | FOREIGN KEY (hotel_id) REFERENCES hotels(hotel_id) ON DELETE RESTRICT | fk_bookings_hotels | RESTRICT |
| booking_rooms | FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE | fk_booking_rooms_bookings | CASCADE |
| booking_rooms | FOREIGN KEY (room_id) REFERENCES rooms(room_id) ON DELETE RESTRICT | fk_booking_rooms_rooms | RESTRICT |
| payments | FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE RESTRICT | fk_payments_bookings | RESTRICT |
| reviews | FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE | fk_reviews_bookings | CASCADE |

## Why these ON DELETE rules

- **RESTRICT** where history matters: a hotel, guest or booking that is still referenced cannot disappear (bookings → hotels, payments → bookings).
- **CASCADE** for composition: a hotel's room types, rooms, amenity links and landmark links are part of the hotel; a booking's booking_rooms are part of the booking.
- **SET NULL** for optional links: if a referrer guest or a home city is removed, the guest stays and the link becomes unknown.
- **Composite FK** rooms(room_type_id, hotel_id) → room_types(room_type_id, hotel_id): a room can never point at another hotel's room type.

# 6. Constraints, triggers, functions and views

## 6.1 Domains

| domain | rule |
|---|---|
| rupees | numeric(12,2), ≥ 0 |
| indian_mobile | 10 digits, first digit 6–9 |
| email_address | regular expression for an email |
| lat_deg / lon_deg | valid latitude / longitude |

## CHECK and EXCLUDE constraints [catalog.checks]

RA / RC: — (catalog)

Rules the database enforces on every INSERT / UPDATE, whatever program sends it.

```
SELECT con.conrelid::regclass AS table_name, con.conname AS constraint_name,
       CASE con.contype WHEN 'c' THEN 'CHECK' WHEN 'x' THEN 'EXCLUDE' END AS kind,
       pg_get_constraintdef(con.oid) AS rule
FROM pg_constraint con JOIN pg_namespace n ON n.oid = con.connamespace
WHERE con.contype IN ('c', 'x') AND n.nspname = 'public' AND con.conrelid <> 0
ORDER BY 1, 2
```

| table_name | constraint_name | kind | rule |
|---|---|---|---|
| states | ck_states_code_format | CHECK | CHECK ((state_code ~ '^[A-Z]{2}$'::text)) |
| cities | ck_cities_metro_tier | CHECK | CHECK (((NOT is_metro) OR (tier = 1))) |
| cities | ck_cities_photo | CHECK | CHECK (((photo_url IS NULL) OR ((photo_url)::text ~ '^https://'::text) AND (photo_credit IS NOT NULL)))) |
| cities | ck_cities_tier | CHECK | CHECK (((tier >= 1) AND (tier <= 3))) |
| landmarks | ck_landmarks_coords | CHECK | CHECK (((latitude IS NULL) = (longitude IS NULL))) |
| landmarks | ck_landmarks_photo | CHECK | CHECK (((photo_url IS NULL) OR ((photo_url)::text ~ '^https://'::text) AND (photo_credit IS NOT NULL)))) |
| hotels | ck_hotels_in_india | CHECK | CHECK (((((latitude)::numeric >= (6)::numeric) AND ((latitude)::numeric <= 37.5)) AND (((longitude)::numeric >= (68)::numeric) AND ((long… |
| hotels | ck_hotels_rating_count | CHECK | CHECK (((review_count >= 0) AND ((avg_rating IS NULL) = (review_count = 0)))) |
| hotels | ck_hotels_rating_range | CHECK | CHECK (((avg_rating >= (1)::numeric) AND (avg_rating <= (5)::numeric))) |
| hotels | ck_hotels_signature | CHECK | CHECK ((is_signature = (source = 'YUPRESERVED'::listing_source_t))) |
| hotels | ck_hotels_star | CHECK | CHECK (((star_rating >= 0) AND (star_rating <= 5))) |
| hotel_landmarks | ck_hotel_landmarks_distance | CHECK | CHECK (((distance_km >= (0)::numeric) AND (distance_km < (100)::numeric))) |
| room_types | ck_room_types_adults | CHECK | CHECK (((max_adults >= 1) AND (max_adults <= 6))) |
| room_types | ck_room_types_area | CHECK | CHECK (((area_sqft >= 50) AND (area_sqft <= 5000))) |
| room_types | ck_room_types_children | CHECK | CHECK (((max_children >= 0) AND (max_children <= 4))) |
| room_types | ck_room_types_price | CHECK | CHECK ((((base_price)::numeric >= (299)::numeric) AND ((base_price)::numeric <= (500000)::numeric))) |
| rooms | ck_rooms_floor | CHECK | CHECK (((floor_no >= 0) AND (floor_no <= 60))) |
| seasons | ck_seasons_dates | CHECK | CHECK ((end_date >= start_date)) |
| seasons | ck_seasons_multiplier | CHECK | CHECK (((multiplier >= 0.50) AND (multiplier <= 3.00))) |
| seasons | ex_seasons_no_overlap | EXCLUDE | EXCLUDE USING gist (daterange(start_date, end_date, '[]'::text) WITH &&) |
| guests | ck_guests_adult | CHECK | CHECK ((date_of_birth <= (((created_at AT TIME ZONE 'Asia/Kolkata'::text))::date - '18 years'::interval))) |

| table_name | constraint_name | kind | rule |
|---|---|---|---|
| guests | ck_guests_not_self | CHECK | CHECK ((referred_by <> guest_id)) |
| bookings | ck_bookings_booked_before_arrival | CHECK | CHECK ((((booked_at AT TIME ZONE 'Asia/Kolkata'::text))::date <= check_in)) |
| bookings | ck_bookings_cancel_after_booking | CHECK | CHECK ((cancelled_at >= booked_at)) |
| bookings | ck_bookings_cancel_state | CHECK | CHECK (((status = 'CANCELLED'::booking_status_t) = (cancelled_at IS NOT NULL))) |
| bookings | ck_bookings_dates | CHECK | CHECK ((check_out > check_in)) |
| bookings | ck_bookings_guests | CHECK | CHECK (((adults >= 1) AND (children >= 0) AND ((adults + children) <= 24))) |
| bookings | ck_bookings_max_stay | CHECK | CHECK (((check_out - check_in) <= 30)) |
| bookings | ck_bookings_ref_format | CHECK | CHECK ((booking_ref ~ '^YR[A-Z0-9]{6}$'::text)) |
| booking_rooms | ck_booking_rooms_amount | CHECK | CHECK (((room_subtotal)::numeric > (0)::numeric)) |
| payments | ck_payments_amount | CHECK | CHECK (((amount)::numeric > (0)::numeric)) |
| payments | ck_payments_card | CHECK | CHECK (((card_last4 IS NULL) OR ((method = 'CARD'::payment_method_t) AND (card_last4 ~ '^[0-9]{4}$'::text)))) |
| payments | ck_payments_upi | CHECK | CHECK (((upi_vpa IS NULL) OR ((method = 'UPI'::payment_method_t) AND ((upi_vpa)::text ~ '^[a-z0-9._-]{2,}@[a-z]{2,}$'::text)))) |
| payments | ck_payments_upi_vpa_required | CHECK | CHECK (((method <> 'UPI'::payment_method_t) OR (upi_vpa IS NOT NULL))) |
| reviews | ck_reviews_rating | CHECK | CHECK (((rating >= 1) AND (rating <= 5))) |

### Triggers [catalog.triggers]

RA / RC: — (catalog)

Event → condition → action rules that run inside the database. A constraint trigger can be DEFERRED to COMMIT time.

```
SELECT t.tgrelid::regclass::text AS table_name, t.tgname AS trigger_name,
       CASE WHEN t.tgconstraint <> 0 THEN 'constraint trigger' ELSE 'trigger' END AS kind,
       substring(pg_get_triggerdef(t.oid) FROM '(BEFORE|AFTER) .* ON') AS fires,
       t.tgfoid::regproc AS function_name
FROM pg_trigger t
JOIN pg_class c ON c.oid = t.tgrelid JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE NOT t.tgisinternal AND n.nspname = 'public'
ORDER BY 1, 2
```

| table_name | trigger_name | kind | fires | function_name |
|---|---|---|---|---|
| booking_rooms | trg_booking_capacity | constraint trigger | AFTER | tg_booking_capacity |
| booking_rooms | trg_booking_rooms_guard | trigger | BEFORE | tg_booking_rooms_guard |
| bookings | trg_bookings_dates | trigger | BEFORE | tg_bookings_dates |
| bookings | trg_bookings_ref | trigger | BEFORE | tg_bookings_ref |
| bookings | trg_bookings_status | trigger | BEFORE | tg_bookings_status |
| payments | trg_payments_refund_guard | trigger | BEFORE | tg_payments_refund_guard |
| reviews | trg_reviews_guard | trigger | BEFORE | tg_reviews_guard |
| reviews | trg_reviews_rollup | trigger | AFTER | tg_reviews_rollup |

## 6.2 Functions and procedures

| routine | type | what it does |
|---|---|---|
| fn_gst_rate(tariff) | function (IMMUTABLE) | GST slab: 0% ≤ ₹1,000 · 5% ≤ ₹7,500 · 18% above |

| routine | type | what it does |
|---|---|---|
| fn_quote_stay(type, in, out, rooms) | set-returning function | night-by-night price: base × season × weekend, then GST |
| fn_stay_total(…) | function | sum of fn_quote_stay |
| fn_room_is_free(room, in, out) | function | overlap test on half-open date ranges |
| fn_available_rooms(hotel, in, out) | set-returning function | free rooms per room type |
| fn_book_stay(…) | function, SECURITY DEFINER | the whole booking as one transaction with row locks |
| sp_cancel_booking(id, OUT refund) | PROCEDURE (CALL) | cancel + refund policy (100% / 50% / 0%) |

## 6.3 Views

| view | purpose |
|---|---|
| v_hotel_cards | hotel + city + state + cheapest price (search cards) |
| v_booking_summary | booking + guest + hotel + rooms + money (front desk, My trips) |
| v_guest_directory | security view: masked phone/email; the only guest data the analyst role can read |
| v_upcoming_arrivals | updatable view WITH CHECK OPTION: edits cannot push a row out of the view |
| mv_city_performance | materialized view: revenue, room nights, ADR per city (REFRESH to update) |

# 7. Functional dependencies and normalization

A functional dependency X → Y means: any two tuples that agree on X also agree on Y. Normalisation removes dependencies that cause redundancy and update, insertion and deletion anomalies. YupReserved was normalised **on the real raw dataset**; every step below exists as tables in schema lab.

## 7.1 Functional dependencies of the final design

| table | functional dependencies | highest NF |
|---|---|---|
| states | state_code → state_name, region, is_ut; state_name → state_code | BCNF |
| cities | city_id → city_name, state_code, lat, lon, is_metro, tier; (city_name, state_code) → city_id | BCNF |
| landmarks | landmark_id → city_id, name, category, lat, lon; (city_id, name) → landmark_id | BCNF |
| hotels | hotel_id → all attributes; source_ref → hotel_id | BCNF |
| hotel_landmarks | (hotel_id, landmark_id) → distance_km | BCNF / 4NF |
| amenities | amenity_id → name, category; name → amenity_id | BCNF |
| hotel_amenities | no non-trivial FD (all-key) | BCNF / 4NF |
| room_types | room_type_id → hotel_id, type_name, bed, capacity, area, base_price; (hotel_id, type_name) → room_type_id | BCNF |
| rooms | room_id → hotel_id, room_number, room_type_id; (hotel_id, room_number) → room_id; room_type_id → hotel_id | 3NF (not BCNF, by choice) |
| seasons | season_id → name, dates, multiplier; name → season_id | BCNF |
| guests | guest_id → all; phone → guest_id; email → guest_id; (id_type, id_number) → guest_id | BCNF |
| bookings | booking_id → guest, hotel, dates, guests, status, …; booking_ref → booking_id; (check_in, check_out) → nights | 3NF* |
| booking_rooms | (booking_id, room_id) → room_subtotal, room_tax | BCNF |
| payments | payment_id → booking_id, kind, method, status, amount, …; gateway_ref → payment_id | BCNF |
| reviews | review_id → booking_id, rating, title, body, created_at; booking_id → review_id | BCNF |

* nights is a **generated column**: the DBMS computes it from (check_in, check_out), so it can never disagree with them. The redundancy is controlled, not an anomaly.

## 7.2 UNF → 1NF → 2NF → 3NF on the raw data

### UNF — the raw listing stores facilities as a list in one cell [nf.unf]

RA / RC: —

hotel_facilities holds a repeating group ("A|B|C|..."): not atomic, so not in 1NF. You cannot index, count or join on one facility.

```
SELECT property_id, property_name, city, left(hotel_facilities, 120) || '…' AS hotel_facilities
FROM lab.unf_listing
WHERE hotel_facilities IS NOT NULL
LIMIT 5
```

| property_id | property_name | city | hotel_facilities |
|---|---|---|---|
| HTLGBO1000016238 | Baragarh Regency | Manali | Doctor on Call\|Dry Cleaning\|Laundry Service Available\|Lobby\|Parking Facilities Available\|Gardens\|Dance Performances (on … |
| HTLGBO1000015822 | Asian Suites A- 585 | Gurgaon | Airport Transfer Available / Surcharge\|Banquet Facilities\|Business Center \|Currency Exchange\|Doctor on Call\|Health Club … |

| property_id | property_name | city | hotel_facilities |
|---|---|---|---|
| HTLGBO10000004551 | Bevvan Resort | Goa | Swimming Pool\|Bar / Lounge \|Laundry Service Available\|Multi Cuisine Restaurant... |

... 2 more row(s) not shown.

## FD violated in the RAW data — city name → state does NOT hold

**[fd.violation_raw]**

RA / RC: σ count(DISTINCT state) > 1 ( city γ ... (staging.goibibo_raw))

In the raw dataset the same city name appears under different state spellings, so 'city' alone cannot determine 'state'. That is why cities has key city_id and candidate key (city_name, state_code).

```
SELECT city, count(DISTINCT state) AS different_state_values, string_agg(DISTINCT state, ' | ') AS values_seen
FROM staging.goibibo_raw
GROUP BY city
HAVING count(DISTINCT state) > 1
ORDER BY 2 DESC, 1
```

| city | different_state_values | values_seen |
|---|---|---|
| Aurangabad | 2 | Bihar \| Maharashtra |
| Bangalore | 2 | KARNATAKA \| Karnataka |
| Chennai | 2 | TAMIL NADU \| Tamil Nadu |
| Delhi | 2 | DELHI \| Delhi |
| Ernakulam | 2 | KERALA \| Kerala |
| Goa | 2 | GOA \| Goa |
| Hyderabad | 2 | ANDHRA PRADESH \| Andhra Pradesh |
| Mumbai | 2 | MAHARASHTRA \| Maharashtra |

**Why this matters.** A decomposition is only lossless if the dependency it splits on really holds. When the walk-through was first run on the uncleaned names, the 3NF re-join produced 357 lost or spurious rows, because 'city → state' is false in the raw file (e.g. Hyderabad filed under both Andhra Pradesh and Telangana). City names are not even unique after cleaning — Aurangabad exists in both Maharashtra and Bihar. So the walk-through (and the real cities table) uses the surrogate city_id, for which city_id → city_name, state holds.

## 1NF — one atomic facility per row (but name and city now repeat) [nf.first]

RA / RC: key = (property_id, facility)

Every cell is atomic. But property_id → property_name, city_id, city, state is a PARTIAL dependency on the key: the name and city repeat once per facility (update anomaly).

```
SELECT * FROM lab.nf1_listing_facility
WHERE property_id = (SELECT property_id FROM lab.nf1_listing_facility LIMIT 1)
ORDER BY facility
```

| operty_id | property_name | city_id | city | state | facility |
|---|---|---|---|---|---|
| LGBO10000057 | Hotel Samdariya inn | 303 | Jabalpur | Madhya Pradesh | 24 Hour Front Desk |
| LGBO10000057 | Hotel Samdariya inn | 303 | Jabalpur | Madhya Pradesh | ATM / Banking |
| LGBO10000057 | Hotel Samdariya inn | 303 | Jabalpur | Madhya Pradesh | Air / Rail Booking |
| LGBO10000057 | Hotel Samdariya inn | 303 | Jabalpur | Madhya Pradesh | Air-Conditioning - Central |
| LGBO10000057 | Hotel Samdariya inn | 303 | Jabalpur | Madhya Pradesh | Baggage Room |
| LGBO10000057 | Hotel Samdariya inn | 303 | Jabalpur | Madhya Pradesh | Banquet Facilities |

… 42 more row(s) not shown.

## 2NF — split off the partial dependency [nf.second]

RA / RC: R → R1(property_id, name, city, state), R2(property_id, facility)

Now every non-key attribute depends on the WHOLE key. But property_id → city_id → city, state is TRANSITIVE: the city and state are repeated for every hotel of that city.

```
SELECT city_id, city, state, count(*) AS hotels_repeating_this_city
FROM lab.nf2_property
GROUP BY city_id, city, state
ORDER BY 3 DESC
LIMIT 10
```

| city_id | city | state | hotels_repeating_this_city |
|---|---|---|---|
| 526 | Durgapur | West Bengal | 2 |
| 531 | Kalimpong | West Bengal | 10 |
| 523 | Dayapur | West Bengal | 1 |
| 533 | Kolkata | West Bengal | 39 |
| 530 | Jalpaiguri | West Bengal | 4 |
| 543 | Tajpur | West Bengal | 1 |

… 4 more row(s) not shown.

## 3NF — city_id → city, state moved to its own table; lossless join check

[nf.third]

RA / RC: R1 ⋈ R2 ⋈ R3 = original 1NF relation

Joining the 3NF tables back gives exactly the 1NF rows — no spurious tuples, nothing lost. That proves the decomposition is lossless.

```
SELECT (SELECT count(*) FROM lab.nf1_listing_facility) AS rows_in_1nf,
       (SELECT count(*) FROM lab.nf3_property p
          JOIN lab.nf3_city c USING (city_id)
          JOIN lab.nf2_property_facility f USING (property_id)) AS rows_after_rejoin,
       (SELECT count(*) FROM (
            SELECT property_id, property_name, city_id, city, state, facility FROM lab.nf1_listing_facility
            EXCEPT
            SELECT p.property_id, p.property_name, p.city_id, c.city, c.state, f.facility
            FROM lab.nf3_property p JOIN lab.nf3_city c USING (city_id)
            JOIN lab.nf2_property_facility f USING (property_id)) d) AS rows_lost_or_spurious
```

| rows_in_1nf | rows_after_rejoin | rows_lost_or_spurious |
|---|---|---|
| 57088 | 57088 | 0 |

## 7.3 BCNF — and why rooms stays in 3NF

## 3NF but not BCNF — rooms (a deliberate choice) [nf.bcnf_rooms]

RA / RC: rooms(room_id, hotel_id, room_number, room_type_id): room_type_id → hotel_id

room_type_id → hotel_id has a determinant that is not a superkey (violates BCNF), but hotel_id is prime (part of candidate key {hotel_id, room_number}), so 3NF holds. Decomposing to BCNF would lose (hotel_id, room_number) → room_id, so we keep 3NF and enforce consistency with a composite FK.

```
SELECT r.room_type_id, count(DISTINCT r.hotel_id) AS hotels_per_room_type
FROM rooms r
GROUP BY r.room_type_id
HAVING count(DISTINCT r.hotel_id) > 1
```

| room_type_id | hotels_per_room_type |
|---|---|

In rooms, room_type_id → hotel_id has a determinant that is not a superkey, so rooms is not in BCNF. It is in 3NF because hotel_id is a **prime attribute** (part of the candidate key {hotel_id, room_number}). The BCNF decomposition R1(room_type_id, hotel_id), R2(room_id,

room_number, room_type_id) is lossless but **not dependency-preserving**: '(hotel_id, room_number) is unique' could no longer be checked in a single table. We therefore keep 3NF and guarantee consistency with the composite foreign key (room_type_id, hotel_id).

## 7.4 4NF and 5NF

### 4NF — two independent multivalued facts blow up into a cross product

**[nf.fourth]**

RA / RC: hotel ↠ amenity | landmark

Storing amenities and landmarks in one table forces every amenity to pair with every landmark. 4NF keeps them in two tables (hotel_amenities, hotel_landmarks).

```
SELECT hotel_name,
       count(*) AS rows_if_combined,
       count(DISTINCT amenity_name) AS amenities,
       count(DISTINCT landmark_name) AS landmarks,
       count(DISTINCT amenity_name) + count(DISTINCT landmark_name) AS rows_in_4nf
FROM lab.mvd_hotel_amenity_landmark
GROUP BY hotel_name
ORDER BY rows_if_combined DESC
```

| hotel_name | rows_if_combined | amenities | landmarks | rows_in_4nf |
|---|---|---|---|---|
| Samudra Mahal | 66 | 11 | 6 | 17 |
| Lutyens Lodge | 64 | 8 | 8 | 16 |
| Pichola Rajbagh | 40 | 10 | 4 | 14 |
| Amber Haveli | 40 | 8 | 5 | 13 |
| Morjim Driftwood Shacks | 36 | 6 | 6 | 12 |
| Charminar Courtyard | 28 | 7 | 4 | 11 |
| Cubbon Leaf | 24 | 6 | 4 | 10 |
| Hooghly Ghat House | 18 | 6 | 3 | 9 |

… 10 more row(s) not shown.

### 5NF — a join dependency: 3-way join is lossless, any 2-way join is not

**[nf.fifth]**

RA / RC: * [ (hotel, plan), (hotel, channel), (plan, channel) ]

Joining only two projections invents spurious rows; joining all three gives back exactly the original 6 rows.

```
SELECT 'original'           AS relation, count(*) AS rows FROM lab.jd_offer
UNION ALL
SELECT 'hp ⋈ hc (2-way)',   count(*) FROM lab.jd_hotel_plan hp JOIN lab.jd_hotel_channel hc USING (hotel)
UNION ALL
SELECT 'hp ⋈ hc ⋈ pc (3-way)', count(*) FROM lab.jd_hotel_plan hp
       JOIN lab.jd_hotel_channel hc USING (hotel)
       JOIN lab.jd_plan_channel pc ON pc.meal_plan = hp.meal_plan AND pc.channel = hc.channel
```

| relation | rows |
|---|---|
| original | 6 |
| hp ⋈ hc (2-way) | 7 |
| hp ⋈ hc ⋈ pc (3-way) | 7 |

# 8. Relational algebra, calculus and SQL

Every query below is stored in 02-database/queries with its relational-algebra (or calculus) form; the outputs are produced live from the database while this report is built.

## Selection (σ) — 5-star hotels in Rajasthan [basics.selection]

RA / RC: σ star_rating = 5 ∧ state_code = 'RJ' (hotels ⋈ cities)

σ keeps the rows that satisfy a predicate. WHERE is the SQL form of σ.

```
SELECT h.hotel_id, h.hotel_name, c.city_name, h.star_rating
FROM hotels h JOIN cities c ON c.city_id = h.city_id
WHERE h.star_rating = 5 AND c.state_code = 'RJ'
ORDER BY c.city_name, h.hotel_name
```

| hotel_id | hotel_name | city_name | star_rating |
|---|---|---|---|
| 146 | Chokhi Dhani - An Ethnic Village Resort | Jaipur | 5 |
| 1277 | TALAI BAGH PALACE | Jaipur | 5 |
| 2216 | The Raj Palace | Jaipur | 5 |
| 2396 | The Tree House | Jaipur | 5 |
| 122 | BRYS Fort | Jaisalmer | 5 |
| 1264 | Sujan-The Serai | Jaisalmer | 5 |
| 3 | Pichola Rajbagh | Udaipur | 5 |

## Projection (π) — the distinct property types on the platform

[basics.projection]

RA / RC: π property_type (hotels)

π keeps chosen columns. Relational algebra works on sets, so duplicates disappear — SQL needs DISTINCT for that.

```
SELECT DISTINCT property_type
FROM hotels
ORDER BY property_type
```

| property_type |
|---|
| Hotel |
| Resort |
| Guest House |
| Homestay |
| Service Apartment |
| Lodge |
| Houseboat |
| Cottage |

… 13 more row(s) not shown.

## Inner (equi) join — hotels with their city and state [joins.inner]

RA / RC: π hotel_name, city_name, state_name (hotels ⋈ hotels.city_id = cities.city_id cities ⋈ cities.state_code = states.state_code states)

Keeps only pairs of rows whose join columns are equal. Every hotel has a city (FK, NOT NULL), so no hotel is lost.

```
SELECT h.hotel_name, c.city_name, s.state_name, s.region
FROM hotels h
INNER JOIN cities c ON c.city_id = h.city_id
INNER JOIN states s ON s.state_code = c.state_code
WHERE h.is_signature
ORDER BY s.region, c.city_name
```

| hotel_name | city_name | state_name | region |
|---|---|---|---|
| Lutyens Lodge | Delhi | Delhi | NORTH |
| Solang Snowline Camp | Manali | Himachal Pradesh | NORTH |
| Ganga Kinara Retreat | Rishikesh | Uttarakhand | NORTH |
| Pine Needle Cottage | Shimla | Himachal Pradesh | NORTH |
| Dal Shikara House | Srinagar | Jammu and Kashmir | NORTH |
| Ghat Ninety-One | Varanasi | Uttar Pradesh | NORTH |
| Vembanad Kettuvallam | Alleppey | Kerala | SOUTH |
| Cubbon Leaf | Bangalore | Karnataka | SOUTH |

… 10 more row(s) not shown.

## Natural join — cities and states on their common column [joins.natural]

RA / RC: cities * states

NATURAL JOIN joins on every column with the same name (here state_code) and shows it once. Convenient, but fragile: adding a same-named column silently changes the join.

```
SELECT city_name, state_code, state_name, region
FROM cities NATURAL JOIN states
WHERE region = 'NORTHEAST'
ORDER BY state_name, city_name
```

| city_name | state_code | state_name | region |
|---|---|---|---|
| Bomdila | AR | Arunachal Pradesh | NORTHEAST |
| Bongaigaon | AS | Assam | NORTHEAST |
| Dibrugarh | AS | Assam | NORTHEAST |
| Golaghat | AS | Assam | NORTHEAST |
| Guwahati | AS | Assam | NORTHEAST |
| Kaziranga | AS | Assam | NORTHEAST |
| Sibsagar | AS | Assam | NORTHEAST |
| Tinsukia | AS | Assam | NORTHEAST |

… 16 more row(s) not shown.

## Theta (non-equi) join — which festival season each night of a stay falls in [joins.theta]

RA / RC: nights ⋈ night ≥ start_date ∧ night ≤ end_date seasons

The join condition is a range (BETWEEN), not equality. This is exactly how fn_quote_stay prices a stay across Diwali.

```
SELECT n.night::date, to_char(n.night, 'Dy') AS day, s.season_name, s.multiplier
FROM generate_series(DATE '2026-11-04', DATE '2026-11-12', INTERVAL '1 day') AS n(night)
JOIN seasons s ON n.night BETWEEN s.start_date AND s.end_date
ORDER BY 1
```

| night | day | season_name | multiplier |
|---|---|---|---|
| 2026-11-06 | Fri | Diwali 2026 | 1.40 |
| 2026-11-07 | Sat | Diwali 2026 | 1.40 |
| 2026-11-08 | Sun | Diwali 2026 | 1.40 |
| 2026-11-09 | Mon | Diwali 2026 | 1.40 |
| 2026-11-10 | Tue | Diwali 2026 | 1.40 |

## Left outer join — every state, even those with no hotels [joins.left_outer]

RA / RC: states ⋈ᴸ cities ⋈ᴸ hotels

Keeps every row of the left table. States with no listed hotels (Lakshadweep, Manipur…) appear with 0 — an inner join would drop them.

```
SELECT s.state_name, s.region, count(h.hotel_id) AS hotels
FROM states s
LEFT JOIN cities c ON c.state_code = s.state_code
LEFT JOIN hotels h ON h.city_id = c.city_id
GROUP BY s.state_code
ORDER BY hotels, s.state_name
```

| state_name | region | hotels |
|---|---|---|
| Lakshadweep | SOUTH | 0 |
| Manipur | NORTHEAST | 0 |
| Arunachal Pradesh | NORTHEAST | 1 |
| Tripura | NORTHEAST | 3 |
| Dadra and Nagar Haveli and Daman and Diu | WEST | 4 |
| Mizoram | NORTHEAST | 4 |
| Nagaland | NORTHEAST | 4 |
| Meghalaya | NORTHEAST | 9 |

… 28 more row(s) not shown.

## Right outer join — every amenity, with how many hotels offer it

**[joins.right_outer]**

RA / RC: hotel_amenities $\bowtie^{R}$ amenities

Mirror image of the left join. Amenities nobody offers yet still appear, with 0.

```
SELECT a.category, a.amenity_name, count(ha.hotel_id) AS hotels
FROM hotel_amenities ha
RIGHT JOIN amenities a ON a.amenity_id = ha.amenity_id
GROUP BY a.amenity_id
ORDER BY hotels, a.amenity_name
```

| category | amenity_name | hotels |
|---|---|---|
| EXPERIENCE | Beachfront | 1 |
| EXPERIENCE | Desert Safari | 1 |
| EXPERIENCE | Fireplace | 1 |
| EXPERIENCE | Guided Walks | 1 |
| EXPERIENCE | Private Deck | 1 |
| EXPERIENCE | Sea View | 1 |
| DINING | Fine Dining | 2 |
| EXPERIENCE | Lake View | 3 |

… 44 more row(s) not shown.

## Full outer join — cities where guests live vs. cities with bookings

**[joins.full_outer]**

RA / RC: home_cities $\bowtie^{f}$ booked_cities

Keeps unmatched rows from BOTH sides: cities that only send travellers, cities that only receive them, and those that do both. (MySQL has no FULL OUTER JOIN — one reason we chose PostgreSQL.)

```
WITH home AS (
    SELECT home_city_id AS city_id, count(*) AS guests_living_here
    FROM guests GROUP BY home_city_id
), visited AS (
    SELECT h.city_id, count(*) AS bookings_received
    FROM bookings b JOIN hotels h ON h.hotel_id = b.hotel_id
    GROUP BY h.city_id
)
SELECT c.city_name,
```

```
       coalesce(home.guests_living_here, 0) AS guests_living_here,
       coalesce(visited.bookings_received, 0) AS bookings_received,
       CASE WHEN home.city_id IS NULL THEN 'only receives'
            WHEN visited.city_id IS NULL THEN 'only sends'
            ELSE 'both' END AS role
FROM home
FULL OUTER JOIN visited ON visited.city_id = home.city_id
JOIN cities c ON c.city_id = coalesce(home.city_id, visited.city_id)
ORDER BY bookings_received DESC, guests_living_here DESC
LIMIT 20
```

| city_name | guests_living_here | bookings_received | role |
|---|---|---|---|
| Delhi | 882 | 7378 | both |
| Bangalore | 873 | 6891 | both |
| Goa | 171 | 4661 | both |
| Chennai | 864 | 3454 | both |
| Mumbai | 848 | 3277 | both |
| Hyderabad | 881 | 3001 | both |
| Pune | 881 | 2957 | both |
| Kolkata | 872 | 2521 | both |

… 12 more row(s) not shown.

## Self join — who referred whom [joins.self]

RA / RC: ρ g(guests) ⋈ g.referred_by = r.guest_id ρ r(guests)

The same table appears twice under two aliases. guests.referred_by is a foreign key to guests itself.

```
SELECT g.guest_id, g.first_name || ' ' || g.last_name AS guest,
       r.guest_id AS referrer_id, r.first_name || ' ' || r.last_name AS referred_by
FROM guests g
JOIN guests r ON r.guest_id = g.referred_by
ORDER BY g.guest_id
LIMIT 15
```

| guest_id | guest | referrer_id | referred_by |
|---|---|---|---|
| 52 | Saanvi Chouhan | 38 | Ayesha Thomas |
| 56 | Aanya Sen | 5 | Ira Parekh |
| 65 | Riya Gowda | 44 | Rashmi Saikia |
| 72 | Jyothi Abraham | 44 | Rashmi Saikia |
| 76 | Sana Gowda | 61 | Ishaan Parekh |
| 86 | Vignesh Yadav | 66 | Harjot Saha |
| 90 | Trisha Pereira | 44 | Rashmi Saikia |
| 98 | Ayesha Desai | 47 | Tanvi Rathore |

… 7 more row(s) not shown.

## Cross join (Cartesian product) — a price calendar [joins.cross]

RA / RC: room_types × nights

Pairs every row with every row (|R| × |S| rows). Useful when you really want all combinations: here every room type of Samudra Mahal × every night of Christmas week.

```
SELECT rt.type_name, n.night::date,
       round(rt.base_price * coalesce(s.multiplier, 1)) AS rate_before_weekend
FROM room_types rt
CROSS JOIN generate_series(DATE '2026-12-21', DATE '2026-12-27', INTERVAL '1 day') AS n(night)
LEFT JOIN seasons s ON n.night BETWEEN s.start_date AND s.end_date
WHERE rt.hotel_id = 1
ORDER BY rt.base_price, n.night
```

| type_name | night | rate_before_weekend |
|---|---|---|
| Harbour Room | 2026-12-21 | 27,750.00 |

| type_name | night | rate_before_weekend |
|---|---|---|
| Harbour Room | 2026-12-22 | 27,750.00 |
| Harbour Room | 2026-12-23 | 27,750.00 |
| Harbour Room | 2026-12-24 | 27,750.00 |
| Harbour Room | 2026-12-25 | 27,750.00 |
| Harbour Room | 2026-12-26 | 27,750.00 |
| Harbour Room | 2026-12-27 | 27,750.00 |
| Gateway View Suite | 2026-12-21 | 51,000.00 |

… 13 more row(s) not shown.

## Semi-join (EXISTS) — hotels that have at least one 5★ review [joins.semi]

RA / RC: hotels ⋉ (bookings ⋈ σ rating = 5 (reviews))

Returns rows of the left table that HAVE a match, without duplicating them per match — unlike a plain join.

```
SELECT h.hotel_name, h.avg_rating, h.review_count
FROM hotels h
WHERE EXISTS (SELECT 1 FROM bookings b JOIN reviews r ON r.booking_id = b.booking_id
              WHERE b.hotel_id = h.hotel_id AND r.rating = 5)
ORDER BY h.review_count DESC
LIMIT 10
```

| hotel_name | avg_rating | review_count |
|---|---|---|
| Samudra Mahal | 4.60 | 155 |
| Lutyens Lodge | 4.50 | 148 |
| Charminar Courtyard | 4.50 | 136 |
| Cubbon Leaf | 4.50 | 118 |
| Hooghly Ghat House | 4.50 | 111 |
| Spice Warehouse Fort Kochi | 4.50 | 73 |
| Amber Haveli | 4.60 | 63 |
| Pichola Rajbagh | 4.40 | 58 |

… 2 more row(s) not shown.

## Anti-join (NOT EXISTS) — hotels that have never been booked [joins.anti]

RA / RC: hotels ▷ bookings (= hotels − (hotels ⋉ bookings))

Rows of the left table with NO match. NOT EXISTS is NULL-safe; NOT IN (subquery) is not (one NULL makes it return nothing).

```
SELECT c.city_name, count(*) AS never_booked_hotels
FROM hotels h JOIN cities c ON c.city_id = h.city_id
WHERE NOT EXISTS (SELECT 1 FROM bookings b WHERE b.hotel_id = h.hotel_id)
GROUP BY c.city_name
ORDER BY never_booked_hotels DESC
LIMIT 10
```

| city_name | never_booked_hotels |
|---|---|
| Alleppey | 1 |
| Deoghar | 1 |

## LATERAL join — the 2 cheapest room types of each signature hotel

[joins.lateral]

RA / RC: — (top-N per group; no single classical operator)

The subquery on the right can refer to the current row on the left, like a for-each loop.

```
SELECT h.hotel_name, cheapest.type_name, cheapest.base_price
FROM hotels h
CROSS JOIN LATERAL (
```

```
    SELECT rt.type_name, rt.base_price FROM room_types rt
    WHERE rt.hotel_id = h.hotel_id ORDER BY rt.base_price LIMIT 2
) AS cheapest
WHERE h.is_signature
ORDER BY h.hotel_id, cheapest.base_price
```

| hotel_name | type_name | base_price |
|---|---|---|
| Samudra Mahal | Harbour Room | 18,500.00 |
| Samudra Mahal | Gateway View Suite | 34,000.00 |
| Charminar Courtyard | Courtyard Room | 3,400.00 |
| Charminar Courtyard | Jharokha Room | 4,600.00 |
| Pichola Rajbagh | Zenana Garden Room | 24,000.00 |
| Pichola Rajbagh | Jharokha Lake Suite | 42,000.00 |
| Morjim Driftwood Shacks | Driftwood Hut | 2,900.00 |
| Morjim Driftwood Shacks | Sea-front Hut | 4,200.00 |

… 28 more row(s) not shown.

## Six-table join — a full booking receipt [joins.multiway]

RA / RC: bookings ⋈ guests ⋈ hotels ⋈ booking_rooms ⋈ rooms ⋈ room_types

Follows the foreign keys from a booking to everything it touches. This is what the 'My trips' page needs.

```
SELECT b.booking_ref, g.first_name || ' ' || g.last_name AS guest, h.hotel_name,
       r.room_number, rt.type_name, b.check_in, b.check_out, b.nights,
       br.room_subtotal, br.room_tax, b.status
FROM bookings b
JOIN guests g        ON g.guest_id = b.guest_id
JOIN hotels h        ON h.hotel_id = b.hotel_id
JOIN booking_rooms br ON br.booking_id = b.booking_id
JOIN rooms r         ON r.room_id = br.room_id
JOIN room_types rt   ON rt.room_type_id = r.room_type_id
WHERE b.hotel_id = 1 AND b.check_in >= DATE '2026-12-01'
ORDER BY b.check_in, r.room_number
LIMIT 15
```

| booking_ref | guest | hotel_name | room_number | type_name | check_in | check_out | nights | room_subtotal | room_tax | status |
|---|---|---|---|---|---|---|---|---|---|---|
| YRFYBF2M | Jacob Trivedi | Samudra Mahal | 110 | Harbour Room | 2026-12-01 | 2026-12-02 | 1 | 18,500.00 | 3,330.00 | CONFIRMED |
| YRQQWSF8 | Aishwarya Rao | Samudra Mahal | 204 | Gateway View Suite | 2026-12-03 | 2026-12-13 | 10 | 353,600.00 | 63,648.00 | CONFIRMED |
| YR8DAFPA | Saurabh Subramanian | Samudra Mahal | 203 | Gateway View Suite | 2026-12-04 | 2026-12-09 | 5 | 176,800.00 | 31,824.00 | CONFIRMED |
| YRTBGYQH | Swati Banerjee | Samudra Mahal | 205 | Gateway View Suite | 2026-12-05 | 2026-12-07 | 2 | 71,400.00 | 12,852.00 | CONFIRMED |
| YRTBGYQH | Swati Banerjee | Samudra Mahal | 208 | Gateway View Suite | 2026-12-05 | 2026-12-07 | 2 | 71,400.00 | 12,852.00 | CONFIRMED |
| YRGAWFWR | Sneha Sangma | Samudra Mahal | 209 | Bunder Presidential Suite | 2026-12-05 | 2026-12-07 | 2 | 151,200.00 | 27,216.00 | CONFIRMED |
| YR66VAXC | Gurpreet Verma | Samudra Mahal | 210 | Bunder Presidential Suite | 2026-12-05 | 2026-12-08 | 3 | 223,200.00 | 40,176.00 | CONFIRMED |
| YR6JQBHF | Gaurav Kulkarni | Samudra Mahal | 108 | Harbour Room | 2026-12-09 | 2026-12-11 | 2 | 37,000.00 | 6,660.00 | CONFIRMED |

… 7 more row(s) not shown.

## UNION — every place name a guest can type into search [sets.union]

RA / RC: π name (cities) ∪ π name (landmarks)

Both inputs must be union-compatible (same number and types of columns). UNION removes duplicates; UNION ALL keeps them and is faster.

```
SELECT city_name AS place, 'city' AS kind FROM cities WHERE city_name ILIKE 'ma%'
UNION
SELECT landmark_name, 'landmark' FROM landmarks WHERE landmark_name ILIKE 'ma%'
ORDER BY place
```

| place | kind |
|---|---|
| Madan Mahal Fort | landmark |
| Madan Mohan temple | landmark |
| Madanmahal Station | landmark |
| Madgaon Railway Station | landmark |
| Madgoan Railway Station | landmark |
| Madhatop | landmark |
| Madikeri | landmark |
| Madikeri Fort | landmark |

... 114 more row(s) not shown.

## INTERSECT — guests who stayed in both Goa and Mumbai [sets.intersect]

RA / RC: π guest_id (σ city='Goa' (...)) ∩ π guest_id (σ city='Mumbai' (...))

Rows present in both results. (MySQL only added INTERSECT in 8.0.31.)

```
SELECT b.guest_id FROM bookings b JOIN hotels h USING (hotel_id) JOIN cities c USING (city_id)
WHERE c.city_name = 'Goa' AND b.status = 'COMPLETED'
INTERSECT
SELECT b.guest_id FROM bookings b JOIN hotels h USING (hotel_id) JOIN cities c USING (city_id)
WHERE c.city_name = 'Mumbai' AND b.status = 'COMPLETED'
ORDER BY guest_id
LIMIT 20
```

| guest_id |
|---|
| 1 |
| 2 |
| 3 |
| 4 |
| 5 |
| 6 |
| 8 |
| 9 |

... 12 more row(s) not shown.

## EXCEPT (set difference) — signature hotels with no pool [sets.except]

RA / RC: π hotel_id (σ is_signature (hotels)) − π hotel_id (hotel_amenities ⋈ σ name='Swimming Pool' (amenities))

Rows in the first result that are not in the second.

```
SELECT h.hotel_id, h.hotel_name FROM hotels h WHERE h.is_signature
EXCEPT
SELECT ha.hotel_id, h.hotel_name FROM hotel_amenities ha
JOIN amenities a USING (amenity_id) JOIN hotels h USING (hotel_id)
WHERE a.amenity_name = 'Swimming Pool'
ORDER BY hotel_id
```

| hotel_id | hotel_name |
|---|---|
| 2 | Charminar Courtyard |
| 4 | Morjim Driftwood Shacks |
| 6 | Vembanad Kettuvallam |
| 7 | Ghat Ninety-One |
| 8 | Chai Bagaan Bungalow |
| 9 | Dal Shikara House |
| 10 | Sonar Qila Haveli |
| 11 | Lutyens Lodge |

… 5 more row(s) not shown.

## GROUP BY + HAVING — cities with more than 50 hotels [agg.group_having]

RA / RC: σ hotels > 50 ( city_name γ count(hotel_id) → hotels, avg(star_rating) (hotels ⋈ cities) )

WHERE filters rows before grouping; HAVING filters groups after.

```
SELECT c.city_name, count(*) AS hotels, round(avg(h.star_rating), 2) AS avg_stars,
       min(v.from_price) AS cheapest_from
FROM hotels h JOIN cities c USING (city_id) JOIN v_hotel_cards v USING (hotel_id)
GROUP BY c.city_name
HAVING count(*) > 50
ORDER BY hotels DESC
```

| city_name | hotels | avg_stars | cheapest_from |
|---|---|---|---|
| Goa | 200 | 0.89 | 499.00 |
| Bangalore | 131 | 1.10 | 499.00 |
| Delhi | 120 | 1.78 | 499.00 |
| Gurgaon | 80 | 1.00 | 899.00 |
| Alleppey | 69 | 0.70 | 899.00 |
| Manali | 68 | 1.47 | 899.00 |
| Chennai | 67 | 1.31 | 499.00 |
| Coorg | 65 | 0.60 | 599.00 |

… 7 more row(s) not shown.

## ROLLUP — revenue by region → state, with subtotals and grand total

[agg.rollup]

RA / RC: — (OLAP extension of γ)

ROLLUP(a, b) = GROUP BY (a,b) ∪ (a) ∪ (). NULL in the output marks a subtotal row.

```
SELECT coalesce(s.region::text, '— ALL REGIONS —') AS region,
       coalesce(s.state_name, '— subtotal —')      AS state,
       count(DISTINCT b.booking_id)                AS bookings,
       sum(br.room_subtotal)                       AS room_revenue
FROM bookings b
JOIN booking_rooms br USING (booking_id)
JOIN hotels h USING (hotel_id) JOIN cities c USING (city_id) JOIN states s USING (state_code)
WHERE b.status = 'COMPLETED'
GROUP BY ROLLUP (s.region, s.state_name)
ORDER BY s.region NULLS LAST, room_revenue DESC NULLS FIRST
```

| region | state | bookings | room_revenue |
|---|---|---|---|
| NORTH | — subtotal — | 13462 | 221,569,841.00 |
| NORTH | Delhi | 5112 | 105,693,721.00 |
| NORTH | Uttar Pradesh | 2136 | 24,761,809.00 |
| NORTH | Uttarakhand | 1470 | 24,305,442.00 |
| NORTH | Haryana | 1588 | 20,029,871.00 |

| region | state | bookings | room_revenue |
|---|---|---|---|
| NORTH | Himachal Pradesh | 1271 | 17,852,163.00 |
| NORTH | Jammu and Kashmir | 696 | 12,395,672.00 |
| NORTH | Chandigarh | 576 | 9,443,579.00 |

... 33 more row(s) not shown.

## CUBE — bookings by status × payment method [agg.cube]

RA / RC: — (OLAP extension of γ)

CUBE(a, b) produces every combination of subtotals: (a,b), (a), (b) and ().

```
SELECT coalesce(b.status::text, 'ALL') AS status,
       coalesce(p.method::text, 'ALL') AS method,
       count(*) AS payments
FROM payments p JOIN bookings b USING (booking_id)
WHERE p.kind = 'CHARGE' AND p.status = 'SUCCESS'
GROUP BY CUBE (b.status, p.method)
ORDER BY b.status NULLS LAST, p.method NULLS LAST
```

| status | method | payments |
|---|---|---|
| CONFIRMED | UPI | 6471 |
| CONFIRMED | CARD | 2992 |
| CONFIRMED | NETBANKING | 1140 |
| CONFIRMED | WALLET | 588 |
| CONFIRMED | ALL | 11191 |
| CHECKED_IN | UPI | 168 |
| CHECKED_IN | CARD | 96 |
| CHECKED_IN | NETBANKING | 32 |

... 25 more row(s) not shown.

## Division (÷) — hotels that have ALL of Pool, Spa and Free Wi-Fi (double NOT EXISTS) [division.not_exists]

RA / RC: π hotel_id, amenity_id (hotel_amenities) ÷ π amenity_id (σ name ∈ {Pool, Spa, Wi-Fi} (amenities))

"Hotels for which there is NO required amenity that they do NOT have." This is the SQL form of the TRC formula {h | ∀a (a ∈ Req → ∃ha (ha.hotel=h ∧ ha.amenity=a))}.

```
SELECT h.hotel_id, h.hotel_name, c.city_name
FROM hotels h JOIN cities c USING (city_id)
WHERE NOT EXISTS (
    SELECT 1 FROM amenities a
    WHERE a.amenity_name IN ('Swimming Pool', 'Spa', 'Free Wi-Fi')
      AND NOT EXISTS (SELECT 1 FROM hotel_amenities ha
                      WHERE ha.hotel_id = h.hotel_id AND ha.amenity_id = a.amenity_id))
ORDER BY c.city_name, h.hotel_name
```

| hotel_id | hotel_name | city_name |
|---|---|---|
| 2047 | Hotel Pushp Villa | Agra |
| 2199 | The Retreat | Agra |
| 202 | BigSplash | Alibag |
| 2655 | The World Backwaters | Alleppey |
| 1335 | Boulevard Riviera | Bangalore |
| 554 | DEEYASUITES | Bangalore |
| 2487 | The Waverly Hotel & Residences | Bangalore |
| 2460 | Urban Stay | Bangalore |

... 88 more row(s) not shown.

## Division (÷) — the same answer with GROUP BY / HAVING COUNT

**[division.count]**

RA / RC: same as division.not_exists

Keep each hotel's rows for the required amenities and accept it only if it matched all 3. Usually faster than double NOT EXISTS.

```
SELECT h.hotel_id, h.hotel_name, c.city_name
FROM hotel_amenities ha
JOIN amenities a USING (amenity_id)
JOIN hotels h USING (hotel_id) JOIN cities c USING (city_id)
WHERE a.amenity_name IN ('Swimming Pool', 'Spa', 'Free Wi-Fi')
GROUP BY h.hotel_id, h.hotel_name, c.city_name
HAVING count(DISTINCT a.amenity_id) = 3
ORDER BY c.city_name, h.hotel_name
```

| hotel_id | hotel_name | city_name |
|---|---|---|
| 2047 | Hotel Pushp Villa | Agra |
| 2199 | The Retreat | Agra |
| 202 | BigSplash | Alibag |
| 2655 | The World Backwaters | Alleppey |
| 1335 | Boulevard Riviera | Bangalore |
| 554 | DEEYASUITES | Bangalore |
| 2487 | The Waverly Hotel & Residences | Bangalore |
| 2460 | Urban Stay | Bangalore |

… 88 more row(s) not shown.

## Scalar subquery — room types priced above their city's average

**[sub.scalar]**

RA / RC: σ base_price > avg_city (room_types ⋈ hotels)

The inner query is re-evaluated for each outer row (correlated) because it refers to h.city_id.

```
SELECT h.hotel_name, c.city_name, rt.type_name, rt.base_price,
       (SELECT round(avg(rt2.base_price)) FROM room_types rt2 JOIN hotels h2 USING (hotel_id)
        WHERE h2.city_id = h.city_id) AS city_avg
FROM room_types rt JOIN hotels h USING (hotel_id) JOIN cities c USING (city_id)
WHERE c.city_name = 'Udaipur'
  AND rt.base_price > (SELECT avg(rt2.base_price) FROM room_types rt2 JOIN hotels h2 USING (hotel_id)
                       WHERE h2.city_id = h.city_id)
ORDER BY rt.base_price DESC
LIMIT 10
```

| hotel_name | city_name | type_name | base_price | city_avg |
|---|---|---|---|---|
| Pichola Rajbagh | Udaipur | Maharani Suite | 95,000.00 | 7,560.00 |
| Pichola Rajbagh | Udaipur | Jharokha Lake Suite | 42,000.00 | 7,560.00 |
| THe Udai bagh | Udaipur | Junior Suite | 28,299.00 | 7,560.00 |
| Hotel Raj Niwas | Udaipur | Super Deluxe Room | 25,499.00 | 7,560.00 |
| The Royal Retreat Resort and Spa | Udaipur | Junior Suite | 24,799.00 | 7,560.00 |
| Pichola Rajbagh | Udaipur | Zenana Garden Room | 24,000.00 | 7,560.00 |
| THe Udai bagh | Udaipur | Premium Room | 20,599.00 | 7,560.00 |
| Hotel Raj Niwas | Udaipur | Deluxe Room | 19,699.00 | 7,560.00 |

… 2 more row(s) not shown.

## > ALL — rooms dearer than every room in Varanasi [sub.any_all]

RA / RC: σ price > max(π price (σ city='Varanasi' …))

x > ALL (S) is true when x beats every element of S; x > ANY (S) when it beats at least one.

```
SELECT h.hotel_name, rt.type_name, rt.base_price
FROM room_types rt JOIN hotels h USING (hotel_id)
WHERE rt.base_price > ALL (SELECT rt2.base_price FROM room_types rt2
                           JOIN hotels h2 USING (hotel_id) JOIN cities c2 USING (city_id)
                           WHERE c2.city_name = 'Varanasi')
ORDER BY rt.base_price
LIMIT 10
```

| hotel_name | type_name | base_price |
|---|---|---|
| Ganga Kinara Retreat | River Cottage | 7,900.00 |
| Emarald Pristine Island | Super Deluxe Room | 7,999.00 |
| Breeze Residency | Super Deluxe Room | 7,999.00 |
| Chumbi Residency | Deluxe Room | 7,999.00 |
| Blueberry Inn | Super Deluxe Room | 7,999.00 |
| Elysium Resorts | Deluxe Room | 7,999.00 |
| sylee tea resort | Super Deluxe Room | 7,999.00 |
| The Willow Resorts | Deluxe Room | 7,999.00 |

… 2 more row(s) not shown.

## CTE (WITH) — each city's busiest hotel [sub.cte]

RA / RC: — (named sub-expressions)

A CTE names an intermediate result so a long query reads top-to-bottom.

```
WITH per_hotel AS (
    SELECT h.city_id, h.hotel_id, h.hotel_name, count(*) AS bookings
    FROM bookings b JOIN hotels h USING (hotel_id)
    WHERE b.status = 'COMPLETED'
    GROUP BY h.hotel_id
), ranked AS (
    SELECT *, rank() OVER (PARTITION BY city_id ORDER BY bookings DESC) AS rnk FROM per_hotel
)
SELECT c.city_name, r.hotel_name, r.bookings
FROM ranked r JOIN cities c USING (city_id)
WHERE r.rnk = 1
ORDER BY r.bookings DESC
LIMIT 15
```

| city_name | hotel_name | bookings |
|---|---|---|
| Mumbai | Samudra Mahal | 396 |
| Delhi | Lutyens Lodge | 369 |
| Hyderabad | Charminar Courtyard | 315 |
| Bangalore | Cubbon Leaf | 302 |
| Kolkata | Hooghly Ghat House | 276 |
| Kochi | Spice Warehouse Fort Kochi | 190 |
| Jaipur | Amber Haveli | 187 |
| Udaipur | Pichola Rajbagh | 179 |

… 7 more row(s) not shown.

## Recursive CTE — the full referral chain above a guest [sub.recursive]

RA / RC: — (transitive closure is NOT expressible in basic relational algebra)

Starts from one guest and repeatedly joins to the referrer until there is none. Classic recursive relationship.

```
WITH RECURSIVE chain AS (
    SELECT guest_id, first_name, last_name, referred_by, 0 AS depth
    FROM guests
    WHERE guest_id = (SELECT g.guest_id FROM guests g
                      JOIN guests p ON p.guest_id = g.referred_by
                      JOIN guests gp ON gp.guest_id = p.referred_by
                      WHERE gp.referred_by IS NOT NULL
                      ORDER BY g.guest_id LIMIT 1)
```

```
  UNION ALL
    SELECT g.guest_id, g.first_name, g.last_name, g.referred_by, c.depth + 1
    FROM guests g JOIN chain c ON g.guest_id = c.referred_by
)
SELECT depth, guest_id, first_name || ' ' || last_name AS guest, referred_by
FROM chain ORDER BY depth
```

| depth | guest_id | guest | referred_by |
|---|---|---|---|
| 0 | 525 | Suresh Mehra | 303 |
| 1 | 303 | Debashish Mukherjee | 72 |
| 2 | 72 | Jyothi Abraham | 44 |
| 3 | 44 | Rashmi Saikia | NULL |

## Tuple relational calculus — guests who reviewed a Udaipur stay with 5★

**[calculus.trc]**

RA / RC: { g.first_name | guests(g) ∧ ∃b ∃r ∃h ∃c (bookings(b) ∧ reviews(r) ∧ hotels(h) ∧ cities(c) ∧ b.guest_id = g.guest_id ∧ r.booking_id = b.booking_id ∧ r.rating = 5 ∧ b.hotel_id = h.hotel_id ∧ h.city_id = c.city_id ∧ c.city_name = 'Udaipur') }

TRC variables range over TUPLES. "∃" becomes EXISTS, "∧" becomes AND. SQL itself is based on TRC.

```
SELECT g.guest_id, g.first_name, g.last_name
FROM guests g
WHERE EXISTS (SELECT 1 FROM bookings b, reviews r, hotels h, cities c
              WHERE b.guest_id = g.guest_id AND r.booking_id = b.booking_id AND r.rating = 5
                AND b.hotel_id = h.hotel_id AND h.city_id = c.city_id AND c.city_name = 'Udaipur')
ORDER BY g.guest_id
LIMIT 15
```

| guest_id | first_name | last_name |
|---|---|---|
| 26 | Pooja | Iyengar |
| 27 | Anirban | Jha |
| 30 | Manish | Abraham |
| 41 | Akash | Malhotra |
| 47 | Tanvi | Rathore |
| 52 | Saanvi | Chouhan |
| 54 | Jyothi | Shetty |
| 65 | Riya | Gowda |

… 7 more row(s) not shown.

## Domain relational calculus — names and stars of Goa beach huts

**[calculus.drc]**

RA / RC: { ⟨n, s⟩ | ∃i ∃t ∃c ( hotels(i, n, t, s, c) ∧ t = 'Beach Hut' ∧ ∃cn ( cities(c, cn) ∧ cn = 'Goa' ) ) }

DRC variables range over attribute VALUES (domains), not whole tuples. Each variable here is a column.

```
SELECT n AS hotel_name, s AS star_rating
FROM (SELECT hotel_id AS i, hotel_name AS n, property_type AS t, star_rating AS s, city_id AS c FROM hotels) AS
    hotels_dom
WHERE t IN ('Beach Hut', 'Cottage')
  AND EXISTS (SELECT 1 FROM (SELECT city_id AS c2, city_name AS cn FROM cities) cities_dom
              WHERE cities_dom.c2 = hotels_dom.c AND cn = 'Goa')
ORDER BY n
```

| hotel_name | star_rating |
|---|---|
| Bhau Cottages | 0 |
| Cabo Cabana Wooden Cottages | 0 |
| Casa D Calma | 0 |
| Cuba Agonda | 2 |

| hotel_name | star_rating |
|---|---|
| Cuba Beach Huts | 0 |
| Morjim Driftwood Shacks | 0 |
| Sunway Cottagesb Goa | 0 |
| Tivai Beach Cottages | 0 |

… 3 more row(s) not shown.

## Window functions — monthly revenue with running total and month-on-month change [sub.window]

RA / RC: — (OLAP)

Window functions compute across related rows without collapsing them, unlike GROUP BY.

```
WITH monthly AS (
    SELECT date_trunc('month', b.check_in)::date AS month, sum(br.room_subtotal) AS revenue
    FROM bookings b JOIN booking_rooms br USING (booking_id)
    WHERE b.status IN ('COMPLETED', 'CHECKED_IN')
    GROUP BY 1
)
SELECT month, revenue,
       sum(revenue) OVER (ORDER BY month)                       AS running_total,
       round(100.0 * (revenue / lag(revenue) OVER (ORDER BY month) - 1), 1) AS mom_change_pct,
       rank() OVER (ORDER BY revenue DESC)                      AS revenue_rank
FROM monthly ORDER BY month
```

| month | revenue | running_total | mom_change_pct | revenue_rank |
|---|---|---|---|---|
| 2025-01-01 | 19,145,395.00 | 19,145,395.00 | NULL | 19 |
| 2025-02-01 | 14,400,834.00 | 33,546,229.00 | -24.80 | 21 |
| 2025-03-01 | 22,272,919.00 | 55,819,148.00 | 54.70 | 17 |
| 2025-04-01 | 19,903,048.00 | 75,722,196.00 | -10.60 | 18 |
| 2025-05-01 | 36,796,085.00 | 112,518,281.00 | 84.90 | 10 |
| 2025-06-01 | 33,425,310.00 | 145,943,591.00 | -9.20 | 13 |
| 2025-07-01 | 12,905,018.00 | 158,848,609.00 | -61.40 | 22 |
| 2025-08-01 | 14,710,060.00 | 173,558,669.00 | 14.00 | 20 |

… 14 more row(s) not shown.

# 9. Retrieval: what the website asks the database

## Price for two nights — Samudra Mahal Harbour Room, 24 → 26 Dec 2026

**[app.quote_two_nights]**

RA / RC: room_types × nights ⋈ᴸ seasons (inside fn_quote_stay)

Night-by-night breakdown: base price × festival multiplier × weekend multiplier, then GST slab per night.

```
SELECT * FROM fn_quote_stay(1, DATE '2026-12-24', DATE '2026-12-26')
```

| night | day_name | season_name | base_price | season_multiplier | weekend_multiplier | nightly_rate | gst_rate | gst_amount | rooms | night_total |
|---|---|---|---|---|---|---|---|---|---|---|
| 2026-12-24 | Thu | Christmas & New Year 2026 | 18,500.00 | 1.50 | 1.00 | 27,750.00 | 0.18 | 4,995.00 | 1 | 32,745.00 |
| 2026-12-25 | Fri | Christmas & New Year 2026 | 18,500.00 | 1.50 | 1.10 | 30,525.00 | 0.18 | 5,494.50 | 1 | 36,019.50 |

## Same room, three different weekends — why prices differ

**[app.quote_compare_dates]**

RA / RC: dates ⋈ fn_stay_total(…)

The same 2-night stay costs more in Diwali and Christmas week (season multiplier) than in the monsoon.

```
SELECT d.label, d.check_in, d.check_in + 2 AS check_out,
       fn_stay_total(1, d.check_in, d.check_in + 2) AS total_incl_gst
FROM (VALUES ('Monsoon weekend', DATE '2026-08-14'),
             ('Ordinary weekend', DATE '2026-11-20'),
             ('Diwali weekend',   DATE '2026-11-06'),
             ('Christmas',        DATE '2026-12-25')) AS d(label, check_in)
ORDER BY total_incl_gst
```

| label | check_in | check_out | total_incl_gst |
|---|---|---|---|
| Monsoon weekend | 2026-08-14 | 2026-08-16 | 40,823.28 |
| Ordinary weekend | 2026-11-20 | 2026-11-22 | 48,026.00 |
| Diwali weekend | 2026-11-06 | 2026-11-08 | 67,236.40 |
| Christmas | 2026-12-25 | 2026-12-27 | 72,039.00 |

## Availability — free rooms per type at Pichola Rajbagh for Diwali

**[app.availability]**

RA / RC: rooms ▷ (booking_rooms ⋈ σ overlaps(stay) ∧ status ≠ CANCELLED (bookings))

A room is free if no live booking overlaps the requested half-open range [check_in, check_out).

```
SELECT * FROM fn_available_rooms(3, DATE '2026-11-07', DATE '2026-11-09')
```

| room_type_id | type_name | base_price | max_adults | max_children | total_rooms | free_rooms |
|---|---|---|---|---|---|---|
| 7 | Zenana Garden Room | 24,000.00 | 2 | 1 | 10 | 7 |
| 8 | Jharokha Lake Suite | 42,000.00 | 3 | 1 | 5 | 5 |
| 9 | Maharani Suite | 95,000.00 | 4 | 2 | 1 | 1 |

## "Near Gateway of India" — hotels within 3 km, nearest first

**[app.near_landmark]**

RA / RC: π … (σ landmark_name ILIKE … ∧ distance_km ≤ 3 (landmarks ⋈ hotel_landmarks ⋈ hotels))

The M:N table hotel_landmarks carries the distance, an attribute of the relationship itself.

```
SELECT h.hotel_name, h.star_rating, hl.distance_km, v.from_price
FROM landmarks l
JOIN hotel_landmarks hl USING (landmark_id)
JOIN hotels h USING (hotel_id)
JOIN v_hotel_cards v USING (hotel_id)
WHERE l.landmark_name ILIKE '%gateway of india%' AND hl.distance_km <= 3
ORDER BY hl.distance_km
LIMIT 15
```

| hotel_name | star_rating | distance_km | from_price |
|---|---|---|---|
| Samudra Mahal | 5 | 0.23 | 18,500.00 |
| Astoria Hotel | 3 | 1.30 | 6,099.00 |
| Traveller's Inn Hotel | 2 | 1.60 | 3,799.00 |
| West End Hotel | 4 | 2.20 | 10,299.00 |
| Hotel Sea Lord | 2 | 2.81 | 3,699.00 |
| Dreamland | 1 | 2.95 | 3,199.00 |

## Mood search — "quiet hill stay" with full-text ranking [app.mood_search]

RA / RC: — (ranked retrieval)

to_tsquery matches word stems (hills → hill) through the GIN index; ts_rank orders by relevance like a search engine.

```
SELECT h.hotel_name, c.city_name, h.property_type,
       round(ts_rank(h.search_doc, q)::numeric, 3) AS relevance
FROM hotels h JOIN cities c USING (city_id),
     websearch_to_tsquery('english', 'quiet hill stay') AS q
WHERE h.search_doc @@ q
ORDER BY relevance DESC, h.avg_rating DESC NULLS LAST
LIMIT 12
```

| hotel_name | city_name | property_type | relevance |
|---|---|---|---|
| Pine Needle Cottage | Shimla | Cottage | 0.79 |
| Chilipili Estate Stay | Coorg | Homestay | 0.66 |
| Daiwik Home Stay | Coorg | Homestay | 0.66 |
| Deepa home stay | Coorg | Homestay | 0.66 |
| Vardaan Home Stay | Shimla | Homestay | 0.66 |
| Sweet Home Stay | Coorg | Homestay | 0.66 |
| yamini home stay | Kasauli | Homestay | 0.66 |
| Tulip Estate Stay | Coorg | Homestay | 0.66 |
| Chingaara Estate Stay | Coorg | Homestay | 0.66 |
| Coorg home stay | Coorg | Homestay | 0.66 |

… 2 more row(s) not shown.

## Typo-tolerant search — "samudra mahl" [app.fuzzy_name]

RA / RC: σ similarity(hotel_name, 'samudra mahl') > 0.3 (hotels)

pg_trgm splits strings into 3-letter pieces; the GIN trigram index finds close matches even with typos.

```
SELECT hotel_name, round(similarity(hotel_name, 'samudra mahl')::numeric, 2) AS similarity
FROM hotels
WHERE hotel_name % 'samudra mahl'
ORDER BY similarity DESC
LIMIT 5
```

| hotel_name | similarity |
|---|---|
| Samudra Mahal | 0.69 |
| HOTEL SAMUDRA | 0.42 |

| hotel_name | similarity |
|---|---|
| Hotel Samudra City | 0.33 |

## Occupancy of signature hotels on Diwali night (8 Nov 2026) [app.occupancy]

RA / RC: hotels γ count(rooms booked) / count(rooms)

Rooms in use on a night = live bookings whose range contains that night.

```
SELECT h.hotel_name,
       count(DISTINCT r.room_id) AS rooms,
       count(DISTINCT br.room_id) AS occupied,
       round(100.0 * count(DISTINCT br.room_id) / count(DISTINCT r.room_id), 1) AS occupancy_pct
FROM hotels h
JOIN rooms r ON r.hotel_id = h.hotel_id
LEFT JOIN booking_rooms br ON br.room_id = r.room_id
     AND EXISTS (SELECT 1 FROM bookings b WHERE b.booking_id = br.booking_id
                 AND b.status <> 'CANCELLED'
                 AND DATE '2026-11-08' >= b.check_in AND DATE '2026-11-08' < b.check_out)
WHERE h.is_signature
GROUP BY h.hotel_id
ORDER BY occupancy_pct DESC
```

| hotel_name | rooms | occupied | occupancy_pct |
|---|---|---|---|
| Lutyens Lodge | 18 | 16 | 88.90 |
| Hooghly Ghat House | 7 | 6 | 85.70 |
| Pine Needle Cottage | 3 | 2 | 66.70 |
| Ghat Ninety-One | 11 | 5 | 45.50 |
| Rue Romain Villa | 7 | 3 | 42.90 |
| Amber Haveli | 15 | 6 | 40.00 |
| Vembanad Kettuvallam | 5 | 2 | 40.00 |
| Charminar Courtyard | 11 | 4 | 36.40 |
| Samudra Mahal | 22 | 7 | 31.80 |
| Spice Warehouse Fort Kochi | 14 | 2 | 14.30 |

… 8 more row(s) not shown.

## Top 10 cities by room revenue (materialized view) [app.top_cities]

RA / RC: π … (mv_city_performance)

Reads precomputed results — instant, but only as fresh as the last REFRESH MATERIALIZED VIEW.

```
SELECT city_name, state_code, hotels, bookings, room_nights, room_revenue, avg_daily_rate
FROM mv_city_performance
ORDER BY room_revenue DESC
LIMIT 10
```

| city_name | state_code | hotels | bookings | room_nights | room_revenue | avg_daily_rate |
|---|---|---|---|---|---|---|
| Delhi | DL | 120 | 5160 | 17,026.00 | 106,868,795.00 | 6,277.00 |
| Mumbai | MH | 52 | 2288 | 7,439.00 | 83,566,533.00 | 11,234.00 |
| Bangalore | KA | 131 | 4858 | 16,138.00 | 72,183,855.00 | 4,473.00 |
| Goa | GA | 200 | 3234 | 10,566.00 | 56,950,761.00 | 5,390.00 |
| Chennai | TN | 67 | 2375 | 7,870.00 | 40,290,805.00 | 5,120.00 |
| Udaipur | RJ | 41 | 989 | 3,284.00 | 38,896,547.00 | 11,844.00 |
| Pune | MH | 57 | 2082 | 6,844.00 | 36,187,856.00 | 5,288.00 |
| Jaipur | RJ | 63 | 1386 | 4,608.00 | 33,691,446.00 | 7,312.00 |
| Kolkata | WB | 41 | 1760 | 5,861.00 | 31,872,169.00 | 5,438.00 |
| Ahmedabad | GJ | 51 | 1760 | 6,002.00 | 30,912,868.00 | 5,150.00 |

## 9.1 'Why can't I…?' — every refusal is a named rule

| attempt | mechanism | database message | rule |
|---|---|---|---|
| Why can't I check out before I check in? | CHECK constraint (tuple-level) | new row for relation "bookings" violates check constraint "ck_bookings_dates" | CHECK constraint: check-out must be after check-in. |
| Why can't I book 45 nights? | CHECK constraint (business limit) | new row for relation "bookings" violates check constraint "ck_bookings_max_stay" | CHECK constraint: a booking can be at most 30 nights. |
| Why can't two guests get the same room on the same night? | Trigger (multi-row rule) | Room 110 is already booked (YRKPS8FJ) for an overlapping stay | TRIGGER trg_booking_rooms_guard: a room must belong to the booking's hotel and be free for those dates. |
| Why can't 9 adults share one Harbour Room? | Deferred constraint trigger (checked at COMMIT) | Booking YRXEWBW2 has 9 adults + 0 children but the rooms hold 2 adults + 1 children | DEFERRED CONSTRAINT TRIGGER trg_booking_capacity: too many guests for the rooms chosen. |
| Why can't I book 3 Maharani Suites? | Stored function with row locks (SELECT … FOR UPDATE) | Only 1 room(s) of this type are free from 2026-12-28 to 2026-12-30 | FUNCTION fn_book_stay: not enough free rooms (rows were locked with SELECT … FOR UPDATE). |
| Why can't a completed stay be moved back to CONFIRMED? | Trigger (state machine) | A COMPLETED booking cannot become CONFIRMED | TRIGGER trg_bookings_status: that status change is not allowed. |
| Why can't I delete a hotel that has bookings? | Foreign key · ON DELETE RESTRICT | update or delete on table "hotels" violates foreign key constraint "fk_bookings_hotels" on table "bookings" | FOREIGN KEY (ON DELETE RESTRICT): the hotel still has bookings. |
| Why is 12345 67890 not accepted as a phone number? | DOMAIN constraint | value for domain indian_mobile violates check constraint "indian_mobile_check" | DOMAIN indian_mobile: 10 digits, starting with 6, 7, 8 or 9. |
| Why can't I sign up with a phone that is already registered? | UNIQUE constraint (candidate key) | duplicate key value violates unique constraint "uq_guests_phone" | UNIQUE (candidate key): this phone number already belongs to another guest. |
| Why can't a 15-year-old create an account? | CHECK constraint using a date expression | new row for relation "guests" violates check constraint "ck_guests_adult" | CHECK constraint: a guest must be at least 18 years old. |
| Why can't I review a stay that hasn't happened yet? | Trigger (cross-table rule) | Booking YRW474CC is CONFIRMED; only completed stays can be reviewed | TRIGGER trg_reviews_guard: only completed stays can be reviewed, after check-out. |
| Why can't I review the same stay twice? | UNIQUE on a foreign key (1:1 relationship) | duplicate key value violates unique constraint "uq_reviews_booking" | UNIQUE: a stay can be reviewed only once (1:1 bookings ↔ reviews). |
| Why can't I add a "Wedding Season" that overlaps Diwali? | EXCLUDE constraint | conflicting key value violates exclusion constraint "ex_seasons_no_overlap" | EXCLUDE constraint: two pricing seasons cannot overlap. |
| Why can't I refund more than the guest paid? | Trigger (aggregate rule) | Refund ₹7385.70 is more than the ₹3692.85 paid for this booking | TRIGGER trg_payments_refund_guard: refund larger than amount paid. |
| Why can't a Mumbai booking use a room in Udaipur? | Trigger + composite key consistency | Room 101 belongs to hotel 3, but booking YR5B3K7J is at hotel 1 | TRIGGER trg_booking_rooms_guard: a room must belong to the booking's hotel and be free for those dates. |
| Why can't the analyst read guests' phone numbers? | DCL · GRANT / REVOKE · security views | permission denied for table guests | PRIVILEGES (GRANT / REVOKE): this database role is not allowed to do that. |

# 10. Query processing and optimization

PostgreSQL translates SQL into a tree of relational-algebra operators and chooses an algorithm for each using cost estimates from table statistics (ANALYZE). The plans below were produced by EXPLAIN (ANALYZE, BUFFERS) on the project data; '@setup' settings disable alternatives so each algorithm can be seen.

## SELECT with no usable index → sequential (linear) scan

Algorithms for SELECT › Linear search

special_request has no index, so every block of bookings is read and each row is tested.

```
SELECT booking_ref FROM bookings WHERE special_request = 'Celebrating anniversary'
```

```
Seq Scan on bookings  (cost=0.00..1796.00 rows=557 width=9) (actual time=0.006..3.720 rows=556 loops=1)
  Filter: ((special_request)::text = 'Celebrating anniversary'::text)
  Rows Removed by Filter: 71446
  Buffers: shared hit=896
Planning:
  Buffers: shared hit=3
Planning Time: 0.026 ms
Execution Time: 3.735 ms
```

operators: Seq Scan · execution 3.535 ms

## SELECT on a key → B-tree index scan

Algorithms for SELECT › Index search

booking_ref has a UNIQUE index: the B-tree is descended in a few page reads instead of scanning 72,000 rows.

```
SELECT * FROM bookings WHERE booking_ref = (SELECT booking_ref FROM bookings WHERE booking_id = 500)
```

```
Index Scan using uq_bookings_ref on bookings  (cost=8.73..16.75 rows=1 width=80) (actual time=0.018..0.019 rows=
    1 loops=1)
  Index Cond: (booking_ref = $0)
  Buffers: shared hit=7
  InitPlan 1 (returns $0)
    ->  Index Scan using pk_bookings on bookings bookings_1  (cost=0.29..8.31 rows=1 width=9) (actual time=0.003
    ..0.003 rows=1 loops=1)
          Index Cond: (booking_id = 500)
          Buffers: shared hit=3
Planning:
  Buffers: shared hit=12
Planning Time: 0.040 ms
Execution Time: 0.026 ms
```

operators: Index Scan · execution 0.006 ms

## SELECT on a range → bitmap index scan

Algorithms for SELECT › Index for range / many rows

Many matching rows: the index builds a bitmap of pages first, then each page is read once in physical order.

```
SELECT booking_ref, status FROM bookings WHERE hotel_id = 1 AND check_in BETWEEN DATE '2026-01-01' AND DATE '202
    6-06-30'
```

```
Bitmap Heap Scan on bookings  (cost=6.61..459.45 rows=182 width=13) (actual time=0.019..0.065 rows=177 loops=1)
  Recheck Cond: ((hotel_id = 1) AND (check_in >= '2026-01-01'::date) AND (check_in <= '2026-06-30'::date))
  Heap Blocks: exact=128
  Buffers: shared hit=131
  ->  Bitmap Index Scan on ix_bookings_hotel_dates  (cost=0.00..6.57 rows=182 width=0) (actual time=0.011..0.011
     rows=177 loops=1)
        Index Cond: ((hotel_id = 1) AND (check_in >= '2026-01-01'::date) AND (check_in <= '2026-06-30'::date))
        Buffers: shared hit=3
Planning Time: 0.023 ms
Execution Time: 0.072 ms
```

operators: Bitmap Heap Scan, Bitmap Index Scan · execution 0.045 ms

## JOIN → hash join (the optimizer's default for big equi-joins)

Algorithms for JOIN › Hash join

Build a hash table on the smaller input (hotels), then probe it once per booking row. O(|R| + |S|).

```
SELECT h.property_type, count(*) FROM bookings b JOIN hotels h ON h.hotel_id = b.hotel_id GROUP BY 1
```

```
HashAggregate  (cost=4094.84..4095.05 rows=21 width=12) (actual time=14.384..14.386 rows=21 loops=1)
  Group Key: h.property_type
  Batches: 1  Memory Usage: 24kB
  Buffers: shared hit=2745
  ->  Hash Join  (cost=1929.62..3734.84 rows=72000 width=4) (actual time=1.240..9.956 rows=72002 loops=1)
        Hash Cond: (b.hotel_id = h.hotel_id)
        Buffers: shared hit=2745
        ->  Seq Scan on bookings b  (cost=0.00..1616.00 rows=72000 width=4) (actual time=0.001..2.246 rows=72002
     loops=1)
              Buffers: shared hit=896
        ->  Hash  (cost=1884.83..1884.83 rows=3583 width=8) (actual time=1.236..1.236 rows=3583 loops=1)
              Buckets: 4096  Batches: 1  Memory Usage: 172kB
              Buffers: shared hit=1849
              ->  Seq Scan on hotels h  (cost=0.00..1884.83 rows=3583 width=8) (actual time=0.003..1.061 rows=35
    83 loops=1)
                    Buffers: shared hit=1849
Planning:
  Buffers: shared hit=13
Planning Time: 0.114 ms
Execution Time: 14.399 ms
```

operators: Aggregate, Hash, Hash Join (Inner), Seq Scan · execution 14.158 ms

## Same JOIN, hash join disabled → sort-merge join

Algorithms for JOIN › Sort-merge join

Both inputs are sorted on hotel_id (or read in index order) and merged in one pass, like merging two sorted lists.
setup: SET LOCAL enable_hashjoin = off; SET LOCAL enable_nestloop = off

```
SELECT h.property_type, count(*) FROM bookings b JOIN hotels h ON h.hotel_id = b.hotel_id GROUP BY 1
```

```
Finalize GroupAggregate  (cost=5940.25..5945.57 rows=21 width=12) (actual time=8.306..9.082 rows=21 loops=1)
  Group Key: h.property_type
  Buffers: shared hit=5931 dirtied=1
  ->  Gather Merge  (cost=5940.25..5945.15 rows=42 width=12) (actual time=8.297..9.069 rows=50 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=5931 dirtied=1
        ->  Sort  (cost=4940.23..4940.28 rows=21 width=12) (actual time=5.675..5.678 rows=17 loops=3)
              Sort Key: h.property_type
              Sort Method: quicksort  Memory: 25kB
              Buffers: shared hit=5931 dirtied=1
              Worker 0:  Sort Method: quicksort  Memory: 25kB
              Worker 1:  Sort Method: quicksort  Memory: 25kB
              ->  Partial HashAggregate  (cost=4939.56..4939.77 rows=21 width=12) (actual time=5.650..5.652 rows
    =17 loops=3)
                    Group Key: h.property_type
                    Batches: 1  Memory Usage: 24kB
                    Buffers: shared hit=5913 dirtied=1
                    Worker 0:  Batches: 1  Memory Usage: 24kB
                    Worker 1:  Batches: 1  Memory Usage: 24kB
                    ->  Merge Join  (cost=2096.64..4789.56 rows=30000 width=4) (actual time=1.809..4.255 rows=24
    001 loops=3)
                          Merge Cond: (b.hotel_id = h.hotel_id)
                          Buffers: shared hit=5913 dirtied=1
                          ->  Parallel Index Only Scan using ix_bookings_hotel_dates on bookings b  (cost=0.29..
    2300.29 rows=30000 width=4) (actual time=0.044..1.063 rows=24001 loops=3)
                                Heap Fetches: 81
                                Buffers: shared hit=356 dirtied=1
                          ->  Sort  (cost=2096.35..2105.31 rows=3583 width=8) (actual time=1.739..1.834 rows=357
    5 loops=3)
                                Sort Key: h.hotel_id
                                Sort Method: quicksort  Memory: 208kB
                                Buffers: shared hit=5557
                                Worker 0:  Sort Method: quicksort  Memory: 208kB
                                Worker 1:  Sort Method: quicksort  Memory: 208kB
                                ->  Seq Scan on hotels h  (cost=0.00..1884.83 rows=3583 width=8) (actual time=0.
    012..1.382 rows=3583 loops=3)
                                      Buffers: shared hit=5547
```

```
Planning:
  Buffers: shared hit=13
Planning Time: 0.067 ms
Execution Time: 9.101 ms
```

operators: Aggregate, Gather Merge, Index Only Scan, Merge Join (Inner), Seq Scan, Sort, Sort Method: quicksort · execution 8.946 ms

## Same JOIN, hash & merge disabled → (index) nested-loop join

Algorithms for JOIN › Nested-loop join

For each outer row, look up the matching inner rows (here through the hotels primary-key index).
setup: SET LOCAL enable_hashjoin = off; SET LOCAL enable_mergejoin = off

```
SELECT h.property_type, count(*) FROM bookings b JOIN hotels h ON h.hotel_id = b.hotel_id GROUP BY 1
```

```
HashAggregate  (cost=5178.06..5178.27 rows=21 width=12) (actual time=20.196..20.197 rows=21 loops=1)
  Group Key: h.property_type
  Batches: 1  Memory Usage: 24kB
  Buffers: shared hit=11639
  ->  Nested Loop  (cost=0.29..4818.06 rows=72000 width=4) (actual time=0.006..16.097 rows=72002 loops=1)
        Buffers: shared hit=11639
        ->  Seq Scan on bookings b  (cost=0.00..1616.00 rows=72000 width=4) (actual time=0.001..2.275 rows=72002
     loops=1)
              Buffers: shared hit=896
        ->  Memoize  (cost=0.29..0.41 rows=1 width=8) (actual time=0.000..0.000 rows=1 loops=72002)
              Cache Key: b.hotel_id
              Cache Mode: logical
              Hits: 68421  Misses: 3581  Evictions: 0  Overflows: 0  Memory Usage: 378kB
              Buffers: shared hit=10743
              ->  Index Scan using pk_hotels on hotels h  (cost=0.28..0.40 rows=1 width=8) (actual time=0.001..0
     .001 rows=1 loops=3581)
                    Index Cond: (hotel_id = b.hotel_id)
                    Buffers: shared hit=10743
Planning Time: 0.058 ms
Execution Time: 20.211 ms
```

operators: Aggregate, Index Scan, Memoize, Nested Loop (Inner), Seq Scan · execution 19.87 ms

## ORDER BY bigger than memory → external merge sort on disk

Algorithms for external sorting

With only 64 kB of sort memory, PostgreSQL writes sorted runs to temporary files and merges them (look for "Sort Method: external merge Disk").
setup: SET LOCAL work_mem = '64kB'

```
SELECT booking_id, booked_at FROM bookings ORDER BY booked_at DESC
```

```
Sort  (cost=11120.86..11300.86 rows=72000 width=12) (actual time=19.197..21.488 rows=72002 loops=1)
  Sort Key: booked_at DESC
  Sort Method: external merge  Disk: 1896kB
  Buffers: shared hit=896, temp read=697 written=766
  ->  Seq Scan on bookings  (cost=0.00..1616.00 rows=72000 width=12) (actual time=0.001..4.914 rows=72002 loops=
    1)
        Buffers: shared hit=896
Planning:
  Buffers: shared hit=6
Planning Time: 0.022 ms
Execution Time: 23.968 ms
```

operators: Seq Scan, Sort, Sort Method: external merge · execution 24.168 ms

## The same ORDER BY with enough memory → quicksort in RAM

Algorithms for external sorting (comparison)

Now the whole input fits: "Sort Method: quicksort Memory". Compare the time with plan.external_sort.
setup: SET LOCAL work_mem = '32MB'

```
SELECT booking_id, booked_at FROM bookings ORDER BY booked_at DESC
```

```
Sort  (cost=7424.86..7604.86 rows=72000 width=12) (actual time=10.125..11.613 rows=72002 loops=1)
  Sort Key: booked_at DESC
  Sort Method: quicksort  Memory: 5885kB
  Buffers: shared hit=896
```

```
  ->  Seq Scan on bookings  (cost=0.00..1616.00 rows=72000 width=12) (actual time=0.001..4.916 rows=72002 loops=
    1)
        Buffers: shared hit=896
Planning Time: 0.014 ms
Execution Time: 13.069 ms
```

operators: Seq Scan, Sort, Sort Method: quicksort · execution 12.547 ms

## ORDER BY … LIMIT → top-N heapsort

Algorithms for sorting › Top-N

Only the best 10 rows are kept in a small heap; the input is never fully sorted.

```
SELECT booking_id, booked_at FROM bookings ORDER BY booked_at DESC LIMIT 10
```

```
Limit  (cost=3171.89..3171.92 rows=10 width=12) (actual time=9.112..9.112 rows=10 loops=1)
  Buffers: shared hit=896
  ->  Sort  (cost=3171.89..3351.89 rows=72000 width=12) (actual time=9.111..9.111 rows=10 loops=1)
        Sort Key: booked_at DESC
        Sort Method: top-N heapsort  Memory: 25kB
        Buffers: shared hit=896
        ->  Seq Scan on bookings  (cost=0.00..1616.00 rows=72000 width=12) (actual time=0.001..4.806 rows=72002
    loops=1)
              Buffers: shared hit=896
Planning Time: 0.021 ms
Execution Time: 9.115 ms
```

operators: Limit, Seq Scan, Sort, Sort Method: top-N heapsort · execution 9.848 ms

## PROJECT with DISTINCT → hash aggregate (duplicate elimination)

Algorithms for PROJECT and set operations

Duplicate elimination: either read guest_id in sorted (index) order and drop repeats (Unique), or use a hash table (HashAggregate). The optimizer picks the cheaper one.

```
SELECT DISTINCT guest_id FROM bookings
```

```
HashAggregate  (cost=1796.00..1960.78 rows=16478 width=4) (actual time=7.618..8.363 rows=20456 loops=1)
  Group Key: guest_id
  Batches: 1  Memory Usage: 1809kB
  Buffers: shared hit=896
  ->  Seq Scan on bookings  (cost=0.00..1616.00 rows=72000 width=4) (actual time=0.001..2.028 rows=72002 loops=1
    )
        Buffers: shared hit=896
Planning Time: 0.018 ms
Execution Time: 8.768 ms
```

operators: Aggregate, Seq Scan · execution 8.902 ms

## INTERSECT → hashed set operation

Algorithms for set operations

Both inputs are appended and a HashSetOp counts which rows came from both sides.

```
SELECT guest_id FROM bookings WHERE hotel_id = 1
INTERSECT
SELECT guest_id FROM bookings WHERE hotel_id = 3
```

```
HashSetOp Intersect  (cost=10.04..1413.44 rows=225 width=8) (actual time=0.302..0.305 rows=14 loops=1)
  Buffers: shared hit=635
  ->  Append  (cost=10.04..1411.38 rows=826 width=8) (actual time=0.025..0.266 rows=821 loops=1)
        Buffers: shared hit=635
        ->  Subquery Scan on "*SELECT* 2"  (cost=10.04..533.53 rows=226 width=8) (actual time=0.025..0.079 rows=
    247 loops=1)
              Buffers: shared hit=221
              ->  Bitmap Heap Scan on bookings  (cost=10.04..531.27 rows=226 width=4) (actual time=0.025..0.069
    rows=247 loops=1)
                    Recheck Cond: (hotel_id = 3)
                    Heap Blocks: exact=218
                    Buffers: shared hit=221
                    ->  Bitmap Index Scan on ix_bookings_hotel_dates  (cost=0.00..9.99 rows=226 width=0) (actual
     time=0.013..0.013 rows=247 loops=1)
                          Index Cond: (hotel_id = 3)
                          Buffers: shared hit=3
        ->  Subquery Scan on "*SELECT* 1"  (cost=16.94..873.72 rows=600 width=8) (actual time=0.046..0.157 rows=
```

```
    574 loops=1)
              Buffers: shared hit=414
              ->  Bitmap Heap Scan on bookings bookings_1  (cost=16.94..867.72 rows=600 width=4) (actual time=0.
    046..0.133 rows=574 loops=1)
                    Recheck Cond: (hotel_id = 1)
                    Heap Blocks: exact=410
                    Buffers: shared hit=414
                    ->  Bitmap Index Scan on ix_bookings_hotel_dates  (cost=0.00..16.79 rows=600 width=0) (actua
    l time=0.025..0.025 rows=574 loops=1)
                          Index Cond: (hotel_id = 1)
                          Buffers: shared hit=4
Planning Time: 0.036 ms
Execution Time: 0.315 ms
```

operators: Append, Bitmap Heap Scan, Bitmap Index Scan, SetOp, Subquery Scan · execution 0.277 ms

## Pipelining — rows stream through operators without temp tables

### Combining operations using pipelining

Scan → join → filter → limit pass rows one at a time. LIMIT stops the whole pipeline early: note "actual rows" far below table size.

```
SELECT b.booking_ref, h.hotel_name
FROM bookings b JOIN hotels h ON h.hotel_id = b.hotel_id
WHERE h.star_rating >= 4
LIMIT 5
```

```
Limit  (cost=0.29..6.24 rows=5 width=28) (actual time=0.014..0.047 rows=5 loops=1)
  Buffers: shared hit=64
  ->  Nested Loop  (cost=0.29..4826.77 rows=4059 width=28) (actual time=0.014..0.047 rows=5 loops=1)
        Buffers: shared hit=64
        ->  Seq Scan on bookings b  (cost=0.00..1616.00 rows=72000 width=13) (actual time=0.001..0.002 rows=21 l
    oops=1)
              Buffers: shared hit=1
        ->  Memoize  (cost=0.29..0.42 rows=1 width=23) (actual time=0.002..0.002 rows=0 loops=21)
              Cache Key: b.hotel_id
              Cache Mode: logical
              Hits: 0  Misses: 21  Evictions: 0  Overflows: 0  Memory Usage: 2kB
              Buffers: shared hit=63
              ->  Index Scan using pk_hotels on hotels h  (cost=0.28..0.41 rows=1 width=23) (actual time=0.002..
    0.002 rows=0 loops=21)
                    Index Cond: (hotel_id = b.hotel_id)
                    Filter: (star_rating >= 4)
                    Rows Removed by Filter: 1
                    Buffers: shared hit=63
Planning:
  Buffers: shared hit=13
Planning Time: 0.074 ms
Execution Time: 0.056 ms
```

operators: Index Scan, Limit, Memoize, Nested Loop (Inner), Seq Scan · execution 0.025 ms

## Heuristic optimization — selection pushed below the join

### Using heuristics in query optimization

We wrote the filter on cities after a 3-way join, but the planner applies σ city_name = 'Udaipur' first (on the cities scan), so the joins handle a few rows instead of thousands.

```
SELECT count(*)
FROM bookings b
JOIN hotels h ON h.hotel_id = b.hotel_id
JOIN cities c ON c.city_id = h.city_id
WHERE c.city_name = 'Udaipur'
```

```
Aggregate  (cost=49.12..49.13 rows=1 width=8) (actual time=0.227..0.227 rows=1 loops=1)
  Buffers: shared hit=129
  ->  Nested Loop  (cost=4.90..48.79 rows=133 width=0) (actual time=0.011..0.195 rows=1359 loops=1)
        Buffers: shared hit=129
        ->  Nested Loop  (cost=4.61..39.49 rows=7 width=4) (actual time=0.009..0.029 rows=41 loops=1)
              Buffers: shared hit=43
              ->  Index Scan using uq_cities_name_state on cities c  (cost=0.28..8.29 rows=1 width=4) (actual ti
    me=0.002..0.002 rows=1 loops=1)
                    Index Cond: ((city_name)::text = 'Udaipur'::text)
                    Buffers: shared hit=3
              ->  Bitmap Heap Scan on hotels h  (cost=4.33..31.13 rows=7 width=8) (actual time=0.006..0.023 rows
```

```
    =41 loops=1)
                  Recheck Cond: (city_id = c.city_id)
                  Heap Blocks: exact=38
                  Buffers: shared hit=40
                  ->  Bitmap Index Scan on ix_hotels_city  (cost=0.00..4.33 rows=7 width=0) (actual time=0.003
    ..0.003 rows=41 loops=1)
                        Index Cond: (city_id = c.city_id)
                        Buffers: shared hit=2
        ->  Index Only Scan using ix_bookings_hotel_dates on bookings b  (cost=0.29..1.12 rows=21 width=4) (actu
    al time=0.002..0.003 rows=33 loops=41)
              Index Cond: (hotel_id = h.hotel_id)
              Heap Fetches: 2
              Buffers: shared hit=86
Planning:
  Buffers: shared hit=25
Planning Time: 0.103 ms
Execution Time: 0.237 ms
```

operators: Aggregate, Bitmap Heap Scan, Bitmap Index Scan, Index Only Scan, Index Scan, Nested Loop (Inner) · execution 0.189 ms

## Distributed fragments — only the relevant "site" is scanned

Unit 5 › Distributed DB (fragment pruning)

The query asks for SOUTH only, so the planner scans lab.site_south and skips the other five fragments.

```
SELECT count(*) FROM lab.bookings_by_region WHERE region = 'SOUTH'
```

```
Aggregate  (cost=564.57..564.58 rows=1 width=8) (actual time=2.086..2.086 rows=1 loops=1)
  Buffers: shared hit=225
  ->  Seq Scan on site_south bookings_by_region  (cost=0.00..507.98 rows=22638 width=0) (actual time=0.007..1.59
    1 rows=22638 loops=1)
        Filter: (region = 'SOUTH'::region_t)
        Buffers: shared hit=225
Planning:
  Buffers: shared hit=85 read=2 dirtied=1
Planning Time: 0.971 ms
Execution Time: 2.089 ms
```

operators: Aggregate, Seq Scan · execution 1.571 ms

## Full-text search → GIN (inverted) index

Unit 5 › Digital libraries (inverted index)

The GIN index maps each word stem to the hotels containing it, so the search does not read every description.

```
SELECT hotel_name FROM hotels WHERE search_doc @@ websearch_to_tsquery('english', 'heritage palace lake')
```

```
Bitmap Heap Scan on hotels  (cost=30.20..34.22 rows=1 width=19) (actual time=0.026..0.026 rows=2 loops=1)
  Recheck Cond: (search_doc @@ '''heritag'' & ''palac'' & ''lake'''::tsquery)
  Heap Blocks: exact=2
  Buffers: shared hit=9
  ->  Bitmap Index Scan on ix_hotels_search_doc  (cost=0.00..30.20 rows=1 width=0) (actual time=0.024..0.024 row
    s=2 loops=1)
        Index Cond: (search_doc @@ '''heritag'' & ''palac'' & ''lake'''::tsquery)
        Buffers: shared hit=7
Planning:
  Buffers: shared hit=1
Planning Time: 0.052 ms
Execution Time: 0.029 ms
```

operators: Bitmap Heap Scan, Bitmap Index Scan · execution 0.01 ms

# 11. Transactions, concurrency control and recovery

ACID: **Atomicity** — fn_book_stay inserts booking, rooms and payment together or not at all. **Consistency** — every constraint and trigger holds at commit (the capacity trigger is deferred to commit time). **Isolation** — row locks and MVCC snapshots keep concurrent bookings apart. **Durability** — the write-ahead log (WAL) is flushed before COMMIT returns. Each demonstration below was run with two real sessions while building this report.

## Double booking race

| t (ms) | session | step | result |
|---|---|---|---|
| 10 | A | BEGIN · fn_book_stay(guest 101, Maharani Suite) → locks the room FOR UPDATE | NULL |
| 22 | A | booking inserted · transaction still open, lock held | {'booking_ref': 'YR73CMBY'} |
| 227 | B | BEGIN · fn_book_stay(guest 202, same room, same dates) → blocked, waiting for A's lock… | NULL |
| 1530 | A | COMMIT · lock released | NULL |
| 1533 | B | lock granted, availability re-checked → REJECTED | Only 0 room(s) of this type are free from 2027-02-15 to 2027-02-17 |
| 1561 | — | clean-up: lab.purge_demo_bookings() removed the demo booking | 1 |

SELECT … FOR UPDATE (an exclusive row lock, two-phase locking) makes B wait until A commits. B then re-checks availability and is refused: the last room can never be sold twice.

## Deadlock

| t (ms) | session | step | result |
|---|---|---|---|
| 8 | A | locked row 1 | NULL |
| 8 | B | locked row 2 | NULL |
| 8 | B | now asks for row 1 (held by the other session)… | NULL |
| 8 | A | now asks for row 2 (held by the other session)… | NULL |
| 1009 | A | ERROR: deadlock detected → chosen as victim, transaction aborted | Process 66227 waits for ShareLock on transaction 1779; blocked by process 66228. Process 66228 waits for ShareLock on transaction 1778; b… |
| 1010 | B | got row 1 once the victim was aborted | NULL |

A waits for B and B waits for A: a cycle in the wait-for graph. After deadlock_timeout (1 s) PostgreSQL detects it and aborts one victim. fn_book_stay prevents this by always locking rooms in room_id order.

## Isolation levels

**READ COMMITTED**

| t (ms) | session | step | result |
|---|---|---|---|
| 11 | Reader | [READ COMMITTED] first read of qty | 10 |
| 13 | Writer | UPDATE qty = qty - 3 · COMMIT | NULL |
| 13 | Reader | second read inside the SAME transaction | 7 |

**REPEATABLE READ**

| t (ms) | session | step | result |
|---|---|---|---|
| 10 | Reader | [REPEATABLE READ] first read of qty | 10 |
| 11 | Writer | UPDATE qty = qty - 3 · COMMIT | NULL |
| 11 | Reader | second read inside the SAME transaction | 10 |

READ COMMITTED sees the other transaction's committed change on the second read (non-repeatable read). REPEATABLE READ reads from a snapshot taken at its first query, so both reads agree.

## Mvcc versions

| t (ms) | session | step | result |
|---|---|---|---|
| 7 | A | row version before (xmin = id of the creating transaction, ctid = physical slot) | {'xmin': '1720', 'xmax': '1779', 'ctid': '(0,2)', 'qty': 4} |
| 8 | A | BEGIN · my transaction id | {'txid': 1784} |
| 8 | A | after UPDATE (uncommitted): a NEW version with my xmin at a new ctid | {'xmin': '1784', 'xmax': '0', 'ctid': '(0,10)', 'qty': 5} |
| 9 | B | meanwhile B still reads the OLD version — no dirty read, and B did not wait | {'xmin': '1720', 'xmax': '1784', 'ctid': '(0,2)', 'qty': 4} |
| 9 | A | ROLLBACK · the new version is never made visible | NULL |

Multiversion concurrency control: a writer creates a new tuple version stamped with its transaction id (a timestamp), readers see the version valid for their snapshot. Readers never block writers, and uncommitted data is never visible.

## Atomic rollback

| t (ms) | session | step | result |
|---|---|---|---|
| 8 | A | bookings before | {'n': 72002} |
| 20 | A | step 1 ✓ booking header inserted (Charminar Courtyard) | NULL |
| 21 | A | step 2 ✗ room 1 is in Samudra Mahal, not Charminar Courtyard | Room 101 belongs to hotel 1, but booking YRXA4WUD is at hotel 2 |
| 23 | A | ROLLBACK · bookings after — step 1 was undone too | {'n': 72002} |

Atomicity: a transaction is all-or-nothing. Step 1 succeeded, step 2 broke a rule, so the whole unit was undone and the database is exactly as it was.

## Crash recovery

./yup recovery commits one row, leaves a second row in an open transaction, stops the server with 'pg_ctl stop -m immediate' (no checkpoint — like a power cut) and restarts it. The server log shows 'database system was not properly shut down; automatic recovery in progress' and 'redo starts at …'. Afterwards the committed row is present (durability) and the uncommitted row is absent (atomicity): REDO replays the WAL, and a transaction without a commit record is never visible.

# 12. Distributed, object-oriented, multimedia databases, data mining and digital libraries

## Horizontal fragmentation — bookings stored per region "site"

**[dist.horizontal]**

RA / RC: bookings = site_north ∪ site_south ∪ site_east ∪ site_west ∪ site_central ∪ site_northeast

Each fragment holds the rows of one region. The union of all fragments is the whole relation (completeness), and no row is in two fragments (disjointness).

```
SELECT tableoid::regclass AS fragment, count(*) AS rows,
       min(check_in) AS first_stay, max(check_in) AS last_stay
FROM lab.bookings_by_region
GROUP BY 1 ORDER BY rows DESC
```

| fragment | rows | first_stay | last_stay |
|---|---|---|---|
| lab.site_west | 22745 | 2025-01-01 | 2027-03-24 |
| lab.site_south | 22638 | 2025-01-01 | 2027-03-24 |
| lab.site_north | 19281 | 2025-01-01 | 2027-03-24 |
| lab.site_east | 4280 | 2025-01-01 | 2027-03-24 |
| lab.site_central | 2146 | 2025-01-01 | 2027-03-23 |
| lab.site_northeast | 910 | 2025-01-01 | 2027-03-19 |

## Vertical fragmentation — guest columns at two sites, rejoined on the key [dist.vertical]

RA / RC: guests = guest_contact ⋈ guest_profile

Each fragment keeps the primary key, so the natural join reconstructs the original rows exactly (lossless).

```
SELECT (SELECT count(*) FROM guests) AS original_rows,
       (SELECT count(*) FROM lab.guest_contact_frag c JOIN lab.guest_profile_frag p USING (guest_id)) AS rejoine
    d_rows
```

| original_rows | rejoined_rows |
|---|---|
| 26001 | 26000 |

## Inheritance — querying the parent class returns all subclasses

**[oodb.inheritance]**

RA / RC: —

lab.heritage_property and lab.floating_property INHERIT lab.property. tableoid::regclass tells which "class" each object belongs to.

```
SELECT tableoid::regclass::text AS stored_in, count(*) AS objects
FROM lab.property GROUP BY 1
UNION ALL
SELECT 'ONLY lab.property', count(*) FROM ONLY lab.property
```

| stored_in | objects |
|---|---|
| lab.property | 2 |
| lab.heritage_property | 14 |
| lab.floating_property | 80 |
| ONLY lab.property | 2 |

## Composite type and a method on it [oodb.composite_type]

RA / RC: —

address is an object with its own attributes (address).city; lab.label(p) behaves like a method.

```
SELECT p.name, (p.address).city AS city, (p.address).state AS state, p.era, lab.label(p) AS label
```

```
FROM lab.heritage_property p
ORDER BY (p.address).state, p.name
LIMIT 12
```

| name | city | state | era | label |
|---|---|---|---|---|
| Balaram Palace Resort | Chitrasani | Gujarat | Royal | Balaram Palace Resort — Chitrasani, Gujarat |
| Spice Warehouse Fort Kochi | Kochi | Kerala | Colonial / Mughal | Spice Warehouse Fort Kochi — Kochi, Kerala |
| Samudra Mahal | Mumbai | Maharashtra | Colonial / Mughal | Samudra Mahal — Mumbai, Maharashtra |
| Amber Haveli | Jaipur | Rajasthan | Colonial / Mughal | Amber Haveli — Jaipur, Rajasthan |
| Fort Dhariyawad | Pratapgarh | Rajasthan | Royal | Fort Dhariyawad — Pratapgarh, Rajasthan |
| Fort Pokaran | Jaisalmer | Rajasthan | Royal | Fort Pokaran — Jaisalmer, Rajasthan |
| Hotel Raj Niwas | Udaipur | Rajasthan | Royal | Hotel Raj Niwas — Udaipur, Rajasthan |
| Hotel Sikar Haveli-Heritage Home | Jaipur | Rajasthan | Royal | Hotel Sikar Haveli-Heritage Home — Jaipur, Rajasthan |

… 4 more row(s) not shown.

## Multimedia — SVG posters stored as binary (bytea) in the database

**[media.posters]**

RA / RC: —

The content column holds the image bytes; metadata (title, MIME type, size) sits alongside. The API serves them at /api/media/{id}.

```
SELECT media_id, hotel_id, title, mime_type, bytes
FROM lab.hotel_media ORDER BY media_id
```

| media_id | hotel_id | title | mime_type | bytes |
|---|---|---|---|---|
| 1 | 15 | Solang Snowline Camp — poster | image/svg+xml | 551 |
| 2 | 6 | Vembanad Kettuvallam — poster | image/svg+xml | 553 |
| 3 | 10 | Sonar Qila Haveli — poster | image/svg+xml | 551 |
| 4 | 9 | Dal Shikara House — poster | image/svg+xml | 550 |
| 5 | 2 | Charminar Courtyard — poster | image/svg+xml | 553 |
| 6 | 1 | Samudra Mahal — poster | image/svg+xml | 544 |
| 7 | 18 | Rue Romain Villa — poster | image/svg+xml | 552 |
| 8 | 17 | Ganga Kinara Retreat — poster | image/svg+xml | 554 |

… 10 more row(s) not shown.

## Association rules — "hotels with a Spa usually also have …"

**[mining.amenity_rules]**

RA / RC: — (market-basket analysis)

support = P(A ∧ B); confidence = P(B | A). Computed by a self-join of hotel_amenities on hotel_id.

```
WITH n AS (SELECT count(DISTINCT hotel_id)::numeric AS hotels FROM hotel_amenities),
```

```
pairs AS (
    SELECT a.amenity_id AS a_id, b.amenity_id AS b_id, count(*) AS both_count
    FROM hotel_amenities a JOIN hotel_amenities b ON a.hotel_id = b.hotel_id AND a.amenity_id <> b.amenity_id
    GROUP BY 1, 2
), single AS (
    SELECT amenity_id, count(*) AS cnt FROM hotel_amenities GROUP BY amenity_id
)
SELECT am_a.amenity_name AS if_hotel_has, am_b.amenity_name AS then_also_has,
       round(p.both_count / n.hotels, 3) AS support,
       round(p.both_count::numeric / s.cnt, 3) AS confidence
FROM pairs p
JOIN single s ON s.amenity_id = p.a_id
JOIN amenities am_a ON am_a.amenity_id = p.a_id
JOIN amenities am_b ON am_b.amenity_id = p.b_id
CROSS JOIN n
WHERE am_a.amenity_name = 'Spa' AND s.cnt >= 20
ORDER BY confidence DESC
LIMIT 10
```

| if_hotel_has | then_also_has | support | confidence |
|---|---|---|---|
| Spa | Room Service 24x7 | 0.13 | 0.94 |
| Spa | Parking | 0.13 | 0.92 |
| Spa | Laundry | 0.12 | 0.89 |
| Spa | Front Desk 24x7 | 0.12 | 0.88 |
| Spa | Restaurant | 0.12 | 0.83 |
| Spa | Car Rental | 0.11 | 0.82 |
| Spa | Doctor on Call | 0.11 | 0.79 |
| Spa | Power Backup | 0.10 | 0.72 |

… 2 more row(s) not shown.

## Travel patterns — which two cities are most often visited by the same guest [mining.city_pairs]

RA / RC: ρ x(trips) ⋈ x.guest = y.guest ∧ x.city < y.city ρ y(trips)

A self-join on guest_id finds every pair of cities one guest has stayed in.

```
WITH trips AS (
    SELECT DISTINCT b.guest_id, c.city_name
    FROM bookings b JOIN hotels h USING (hotel_id) JOIN cities c USING (city_id)
    WHERE b.status = 'COMPLETED'
)
SELECT x.city_name AS city_a, y.city_name AS city_b, count(*) AS shared_guests
FROM trips x JOIN trips y ON x.guest_id = y.guest_id AND x.city_name < y.city_name
GROUP BY 1, 2
ORDER BY shared_guests DESC
LIMIT 10
```

| city_a | city_b | shared_guests |
|---|---|---|
| Bangalore | Delhi | 1039 |
| Bangalore | Goa | 776 |
| Delhi | Goa | 773 |
| Bangalore | Chennai | 628 |
| Chennai | Delhi | 621 |
| Delhi | Mumbai | 602 |
| Bangalore | Mumbai | 587 |
| Bangalore | Hyderabad | 552 |

… 2 more row(s) not shown.

**Digital libraries / information retrieval:** hotels.search_doc is a generated tsvector (weighted name, locality, tagline and description) with a GIN inverted index; 'quiet hill stay' is matched by word stems and ranked with ts_rank (section 9). A trigram index gives typo-tolerant name

search.

# 13. Interfaces: API, CLI and website

| method | endpoint | SQL / object used |
|---|---|---|
| GET | /api/destinations?q= | UNION of cities, landmarks, hotels with trigram similarity |
| GET | /api/search?… | joins + full-text + division (must-have amenities) + fn_available_rooms |
| GET | /api/hotels/{id} | hotels ⋈ cities ⋈ states, amenities, landmarks, room types, reviews |
| GET | /api/hotels/{id}/availability | fn_available_rooms, fn_stay_total |
| GET | /api/quote | fn_quote_stay (night-by-night price) |
| POST | /api/auth/otp, /api/auth/verify | lookup by candidate key guests.phone (OTP mocked) |
| POST | /api/guests | INSERT guests (domains, UNIQUE, CHECK age) |
| POST | /api/bookings | fn_book_stay (locks, triggers, payment) in one transaction |
| POST | /api/bookings/{ref}/cancel | CALL sp_cancel_booking |
| POST | /api/bookings/{ref}/review | INSERT reviews (trg_reviews_guard, trg_reviews_rollup) |
| GET/POST | /api/lab/… | catalog browser, named queries, EXPLAIN, why-not, concurrency |

## 13.1 CLI

```
./yup tables                        15 tables, rows, PK, FK/UNIQUE/CHECK counts
./yup show hotels -w "star_rating = 5" -n 10
./yup describe bookings             columns, constraints, referenced-by, indexes, triggers
./yup keys                          primary, candidate and foreign keys
./yup queries -t join               list named queries        ./yup q joins.full_outer
./yup plans                         ./yup explain plan.external_sort
./yup why                           run all 16 constraint scenarios   ./yup why why.double_booking
./yup txn deadlock                  (double_booking_race | isolation_levels | mvcc_versions | atomic_rollback)
./yup quote 1 2026-12-24 2026-12-26
./yup find "gateway"                ./yup sql "SELECT count(*) FROM bookings"
./yup recovery                      crash-recovery demonstration
```

## 13.2 Website

Home (search by city, landmark or mood; signature stays; festival price calendar) → Search results (filters in the URL, compare up to three stays) → Hotel page (story, neighbourhood with walking times, rooms priced for your dates, night-by-night breakdown, reviews) → Checkout (phone + OTP sign-in, UPI first, honest price) → Confirmation (booking reference) → My trips (cancel with refund preview, write a review) → Under the hood (tables, query library, plans, why-not, transactions). English / □□□□□ and light / dark themes; motion respects the reduce-motion setting; all imagery is lightweight SVG drawn in code.

# 14. How to run

```
scripts/db.sh start                         # PostgreSQL on port 5433
scripts/db.sh build                         # (re)create the database from 01-dataset/processed (~3 min)
cd 03-backend && ~/.yupreserved/venv/bin/uvicorn app.main:app --port 8000
cd 05-frontend && npm run dev               # http://localhost:5173
./yup tables                                # CLI
python3 01-dataset/pipeline/build_dataset.py            # regenerate the dataset (optional)
~/.yupreserved/venv/bin/python 06-docs/report/build_report.py   # rebuild this PDF
```

# Part C · Viva questions and answers

## Design and ER model

### Why did you choose a hotel reservation system?

It needs almost every DBMS idea for real reasons: many-to-many relationships (hotels ↔ amenities, bookings ↔ rooms), a weak entity (rooms), date-range rules no CHECK can express (double booking), money that must be exact, and concurrency (two people booking the last room).

### Why exactly these 15 tables?

Each is one entity or one M:N relationship of the ER model: 12 entities (states, cities, landmarks, hotels, amenities, room_types, rooms, seasons, guests, bookings, payments, reviews) and 3 M:N relationship tables (hotel_amenities, hotel_landmarks, booking_rooms). Nothing is stored twice except controlled derived values.

### Why is ROOM a weak entity?

A room number like '101' means nothing on its own — every hotel has a room 101. It is identified only together with its hotel: room_number is a **partial key**, and (hotel_id, room_number) identifies the room. In the table we keep that as a UNIQUE candidate key and add a surrogate room_id as primary key.

### What is the cardinality of bookings and rooms?

M:N. A booking can hold several rooms (a family books 2), and a room appears in many bookings over time. Hence booking_rooms with composite PK (booking_id, room_id) and the relationship attributes room_subtotal and room_tax.

### Where is a 1:1 relationship?

bookings ↔ reviews: reviews.booking_id is a foreign key AND UNIQUE, so a stay has at most one review.

### Where is a recursive relationship?

guests.referred_by references guests.guest_id ('refers'). The self join and the recursive CTE (sub.recursive) use it.

### Which attributes are derived and how are they handled?

bookings.nights is a GENERATED column (check_out − check_in); age is computed in v_guest_directory; hotels.avg_rating and review_count are stored for speed and kept correct by trigger trg_reviews_rollup.

## Keys

### Why is state_code a natural primary key but hotels use a surrogate key?

State codes (MH, RJ) are short, stable and universally known, so they make good natural keys. Hotel names are not unique and can change; source_ref is long text. A surrogate integer is small, never changes and keeps joins fast.

### Name a super key, a candidate key and an alternate key in guests.

Super key: {guest_id, phone}. Candidate keys: {guest_id}, {phone}, {email}, {id_type, id_number}. Primary key: guest_id. Alternate keys: phone, email and (id_type, id_number), each enforced with UNIQUE.

### Why is (id_type, id_number) a composite key?

Neither column is unique alone — two guests can both use PAN, and a passport and a PAN could share digits — but the pair identifies one document.

### Why does rooms have UNIQUE (room_type_id, hotel_id) in room_types?

So that rooms can declare the composite foreign key (room_type_id, hotel_id) → room_types. That FK guarantees a room's type belongs to the same hotel as the room; a single-column FK could not.

### Why does booking_rooms NOT contain hotel_id?

With key (booking_id, room_id), hotel_id would depend on booking_id alone — a partial dependency (2NF violation). The trigger trg_booking_rooms_guard checks that the room's hotel equals the booking's hotel instead.

### Why ON DELETE RESTRICT for bookings → hotels but CASCADE for room_types → hotels?

Bookings and payments are history and money: deleting a hotel must not silently erase them. Room types, rooms and amenity links are part of the hotel (composition), so they go with it — but only if no booking still uses a room.

## Constraints and triggers

### Why can't I enter a check-out date before the check-in date?

CHECK constraint ck_bookings_dates (check_out > check_in). The DBMS rejects the row whatever program sends it.

### Why is double booking prevented by a trigger and not a CHECK?

A CHECK sees only the row being inserted. 'Is this room already taken for an overlapping range?' needs OTHER rows of booking_rooms and bookings, so it is a BEFORE INSERT trigger (trg_booking_rooms_guard).

### How do you test whether two stays overlap?

Stays are half-open ranges [check_in, check_out). Two ranges [a,b) and [c,d) overlap exactly when a < d AND c < b. So checking out and another guest checking in on the same day is allowed.

### What is a deferred constraint trigger and why use one?

trg_booking_capacity is DEFERRABLE INITIALLY DEFERRED: it runs at COMMIT. A booking for 5 adults inserts its rooms one by one; checking after the first room would wrongly fail. At commit all rooms exist and capacity can be judged.

### What does the EXCLUDE constraint on seasons do?

It uses a GiST index on daterange(start_date, end_date) with the overlap operator so that no two seasons share a night; otherwise a night would have two multipliers.

### Your photo CHECK first rejected cities without a photo. Why?

CHECK (photo_url ~ '^https://' AND photo_credit IS NOT NULL): for a NULL photo the regex gives UNKNOWN and the credit test gives FALSE; UNKNOWN AND FALSE = FALSE, so the row was rejected (a CHECK fails only on FALSE, passes on TRUE or UNKNOWN). The fix: CHECK (photo_url IS NULL OR (photo_url ~ '^https://' AND photo_credit IS NOT NULL)).

### How is the OTP stored securely?

Never in plain text: auth.otp_challenges keeps an HMAC-SHA256 of (phone, code) with a server secret, an expiry time and an attempt counter (CHECK attempts BETWEEN 0 AND 5). Verification locks the row FOR UPDATE and compares hashes in constant time.

### What is a domain? Give an example.

A named type with a rule: indian_mobile is varchar(10) CHECK (VALUE ~ '^[6-9][0-9]{9}$'). Every phone column uses it.

### Why can't a completed booking go back to CONFIRMED?

trg_bookings_status implements a state machine: PENDING → CONFIRMED → CHECKED_IN → COMPLETED, plus CANCELLED / NO_SHOW. Any other change raises an error, so history cannot be rewritten.

# Normalization

### Where did 1NF fail in your raw data?

hotel_facilities held 'Parking|Room Service|Wi-Fi|...' in one cell — a repeating group. lab.nf1_listing_facility makes one row per (hotel, facility); the final design is hotel_amenities.

### Show a partial dependency you removed (2NF).

In nf1_listing_facility(property_id, facility, property_name, city, state) with key (property_id, facility), property_id → property_name, city, state depends on part of the key. Split into nf2_property and nf2_property_facility.

### Show a transitive dependency you removed (3NF).

property_id → city → state. The state was repeated for every hotel of a city; nf3_city(city, state) removes it. In the final schema: hotels.city_id → cities.state_code.

### How do you prove the decomposition is lossless?

Query nf.third rejoins the 3NF tables and compares with the 1NF table using EXCEPT: zero rows lost, zero spurious. Theory: each split shares a key of one side (R1 ∩ R2 → R1 or R1 ∩ R2 → R2).

### What happens if you decompose on an FD that does not hold?

The join is lossy. On the raw names, city → state is false (Hyderabad under two states), and the 3NF re-join gave 357 lost/spurious rows. City names are not unique either (Aurangabad: Maharashtra and Bihar), which is why cities has the surrogate key city_id and the candidate key (city_name, state_code).

### Is every table in BCNF?

All except rooms, deliberately. room_type_id → hotel_id has a non-superkey determinant. rooms is still 3NF because hotel_id is prime. A BCNF split would lose the dependency (hotel_id, room_number) → room_id, so we keep 3NF and a composite FK.

### What is a multivalued dependency here (4NF)?

hotel ↠ amenity and hotel ↠ landmark are independent. Storing both in one table forces every amenity to pair with every landmark (nf.fourth shows the blow-up). 4NF keeps them in hotel_amenities and hotel_landmarks.

### What is a join dependency (5NF)?

lab.jd_offer(hotel, meal_plan, channel) equals the join of its three binary projections, but the join of any two creates spurious rows (nf.fifth). That 3-way join dependency is what 5NF is about.

### Why do you store room_subtotal if prices can be calculated?

It is not derivable later: base prices and seasons can change, but the price agreed at booking time is a fact of that booking. It depends on the whole key (booking_id, room_id).

# Queries, algebra and calculus

### How do you retrieve the price for two nights from date X to date Y?

SELECT * FROM fn_quote_stay(room_type_id, X, Y): a CROSS JOIN of the room type with generate_series(nights), a THETA LEFT JOIN to seasons (night BETWEEN start AND end), × weekend factor, then the GST slab per night.

### Which join returns states that have no hotels?

LEFT OUTER JOIN states → cities → hotels (joins.left_outer): Lakshadweep and Manipur appear with 0.

### Write division in SQL.

Hotels that have ALL of {Pool, Spa, Wi-Fi}: double NOT EXISTS — 'no required amenity that the hotel lacks' (division.not_exists), or GROUP BY hotel HAVING count(DISTINCT amenity) = 3 (division.count).

### What is the difference between TRC and DRC?

In tuple relational calculus variables range over whole tuples ({g | guests(g) ∧ ∃b …}); in domain relational calculus they range over attribute values ({⟨n, s⟩ | ∃i, t, c hotels(i, n, t, s, c) ∧ …}). SQL's EXISTS is TRC.

### Semi-join vs inner join?

An inner join repeats a hotel once per matching review; a semi-join (EXISTS) returns each hotel once if any match exists.

### Why NOT EXISTS and not NOT IN for the anti-join?

If the subquery returns a NULL, x NOT IN (…) is UNKNOWN for every row and returns nothing; NOT EXISTS is NULL-safe.

# Query processing

### Which join algorithm does PostgreSQL use for bookings ⋈ hotels and why?

Hash join: build a hash table on the smaller input (3,583 hotels) and probe it with 72,000 bookings — one pass over each, O(|R| + |S|). With hash joins disabled it chooses a sort-merge join, and with both disabled a nested loop.

### Show external sorting.

plan.external_sort sets work_mem = 64 kB; the plan reports 'Sort Method: external merge Disk: … kB' — sorted runs written to temp files and merged. With 32 MB it becomes an in-memory quicksort.

### What is pipelining?

Operators pass tuples to their parent one at a time without materialising intermediate tables. In plan.pipelining LIMIT 5 stops the scan and join after a handful of rows.

### Give an example of heuristic optimization.

Selection pushdown: in plan.heuristic_pushdown the filter city_name = 'Udaipur' is applied on the cities scan before the joins, so the joins process a few rows instead of thousands.

### Why a GIN index on search_doc?

A GIN index is an inverted index (word → hotels containing it), so full-text search does not read every description.

# Transactions, concurrency and recovery

### What happens if two guests book the last room at the same time?

fn_book_stay locks the room rows with SELECT … FOR UPDATE. The second transaction waits; when the first commits, the second re-checks availability and is refused ('Only 0 room(s) … are free'). The race demo shows the 1.3 s wait.

### How do you avoid deadlocks in booking?

Rooms are always locked in room_id order, so two transactions can never hold locks in opposite orders. The deadlock demo shows what happens otherwise: PostgreSQL detects the cycle after deadlock_timeout and aborts one victim.

### Is PostgreSQL lock-based or timestamp-based?

Both ideas: writers take row locks (two-phase locking), while reads use MVCC — each row version carries transaction ids (xmin / xmax), a form of timestamp, and each statement or transaction reads a snapshot. Readers never block writers.

### Show a non-repeatable read.

In READ COMMITTED a second read in the same transaction sees another session's committed update; in REPEATABLE READ both reads return the same value (txn isolation_levels).

### How does the database recover from a crash?

Write-ahead logging: every change is written to the WAL before it is applied, and COMMIT waits for the WAL flush. After a crash PostgreSQL REDOes the log from the last checkpoint; transactions without a commit record stay invisible. ./yup recovery demonstrates it.

### Why is fn_book_stay SECURITY DEFINER?

The app role yup_app may not UPDATE or lock rooms directly. The function runs with its owner's rights, so the only way the app can lock rooms is through this one audited booking routine — least privilege.

## Dataset, system and Unit 5

### Where does your data come from and is it real?

Hotel listings are real: the Goibibo dataset (4,000 Indian properties, Kaggle/PromptCloud). Prices, guests, bookings and reviews are simulated with a fixed seed because no public dataset contains them; the 18 Signature hotels are original. All of this is documented in 01-dataset/README.md.

### Why did you choose PostgreSQL over MySQL?

The installed MySQL 5.7 ignores CHECK constraints and lacks FULL OUTER JOIN, INTERSECT/EXCEPT, CTEs and window functions. PostgreSQL also gives EXCLUDE, deferred constraints, inheritance, partitioning and full-text search.

### How is a distributed database shown?

lab.bookings_by_region is horizontally fragmented by region into six partitions ('sites'); the planner prunes fragments (plan.partition_pruning). Vertical fragments of guests rejoin losslessly on the key.

### How is an object-oriented database shown?

A composite type lab.address_t (an object with attributes), tables that INHERIT from lab.property (IS-A), and a function lab.label(p) acting as a method.

### How is a multimedia database shown?

lab.hotel_media stores SVG posters as bytea with MIME type and size; the API streams them and the hotel page displays them.

### Where is data mining?

Association rules on amenities (support and confidence via a self-join), co-visited city pairs, ROLLUP / CUBE and window functions.

# Part D · Syllabus coverage

## Unit 1 · Introduction

| topic | where it is in YupReserved |
|---|---|
| Concept and goals of DBMS | Part B §1: data integrity, sharing, security, independence achieved by PostgreSQL |
| Database languages | DDL (schema/*.sql), DML (seed, queries), DCL (07_roles.sql), TCL (BEGIN/COMMIT/ROLLBACK in fn_book_stay, lab) |
| Database users | roles postgres (DBA), yup_app (application), yup_analyst (end user via views), yup_lab |
| Database abstraction | physical (files, indexes), logical (15 tables), view level (v_hotel_cards, v_guest_directory) |
| Database architecture | three-tier architecture (Part B §2); client–server PostgreSQL |
| Relational model and constraints | 02_tables.sql: domain, key, entity-integrity, referential-integrity, CHECK, EXCLUDE |
| ER model, relationship sets, keys, mapping, design | Part B §4: Chen diagram, weak entity, cardinalities, ER → relational mapping table |

## Unit 2 · Relational model

| topic | where it is in YupReserved |
|---|---|
| The catalog | catalog.* queries (pg_catalog / information_schema), ./yup describe, Lab 'Tables' tab |
| Types | ENUM types and DOMAINs (01_types.sql), composite type lab.address_t |
| Keys | catalog.primary_keys / candidate_keys / foreign_keys; Part B §5 |
| Relational algebra (fundamental + additional operations) | σ π ρ ∪ − × in basics.*, sets.*, joins.cross; ⋈ ÷ ∩ γ in joins.*, division.*, agg.* |
| Tuple and domain relational calculus | calculus.trc, calculus.drc, division.not_exists |
| SQL fundamentals | 66 named queries: predicates, CASE, subqueries, CTEs, recursion, window functions |
| Integrity | constraints + 16 why-not scenarios |
| Triggers | 8 triggers incl. a deferred constraint trigger (04_triggers.sql) |
| Views | 5 views incl. updatable WITH CHECK OPTION and a materialized view |

## Unit 3 · Relational database design

| topic | where it is in YupReserved |
|---|---|
| Normalization using functional dependencies | Part B §7: FD table; UNF → 1NF → 2NF → 3NF on the real data (lab.nf*) |
| BCNF and dependency preservation | nf.bcnf_rooms — rooms deliberately 3NF |
| Multivalued dependency (4NF) | nf.fourth (lab.mvd_hotel_amenity_landmark) |
| Join dependency (5NF) | nf.fifth (lab.jd_offer) |
| Lossless decomposition | nf.third and dist.vertical |

## Unit 4 · Query processing, optimization and tuning

| topic | where it is in YupReserved |
|---|---|
| Translating SQL into relational algebra | every named query carries its RA form; EXPLAIN shows the operator tree |
| External sorting | plan.external_sort vs plan.in_memory_sort, plan.top_n |

| topic | where it is in YupReserved |
|---|---|
| SELECT algorithms | plan.seq_scan, plan.index_scan, plan.bitmap_scan |
| JOIN algorithms | plan.hash_join, plan.merge_join, plan.nested_loop |
| PROJECT and set operations | plan.project_distinct, plan.set_operation |
| Pipelining | plan.pipelining |
| Heuristic optimization | plan.heuristic_pushdown |
| Database tuning | 05_indexes.sql: FK, composite, partial, GIN, trigram and expression indexes; laptop-tuned settings |

# Unit 5 · Transactions, concurrency, recovery and new applications

| topic | where it is in YupReserved |
|---|---|
| Transaction processing concepts | ACID in fn_book_stay; txn atomic_rollback |
| Lock-based protocols | SELECT … FOR UPDATE; txn double_booking_race |
| Timestamp-based protocols | MVCC transaction ids xmin/xmax; txn mvcc_versions |
| Deadlock handling | txn deadlock (detection + victim); ordered locking prevents it in bookings |
| Crash recovery | WAL; ./yup recovery |
| Distributed database | horizontal + vertical fragmentation (lab.bookings_by_region, guest fragments) |
| Object-oriented database | composite types, table inheritance, method function (lab.*) |
| Multimedia database | lab.hotel_media (bytea), served at /api/lab/media/{id} |
| Data mining | mining.amenity_rules, mining.city_pairs, agg.rollup, agg.cube, sub.window |
| Digital libraries | full-text search (tsvector + GIN), app.mood_search, trigram search |